Verifikation verteilter Systeme in Isabelle

Ein Vergleich von Kleppmann und I/O-Automaten

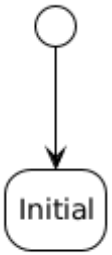
Sepp Stage

TU Chemnitz, Fakultät Betriebssysteme

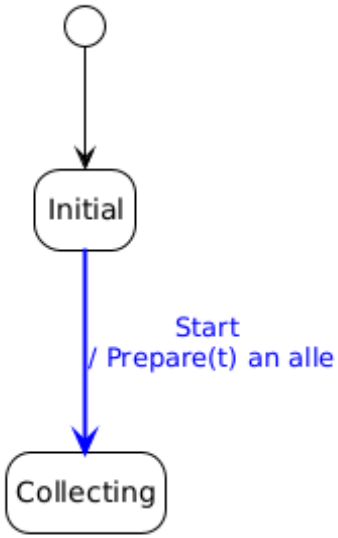
TECHNISCHE UNIVERSITÄT
CHEMNITZ

AT&T

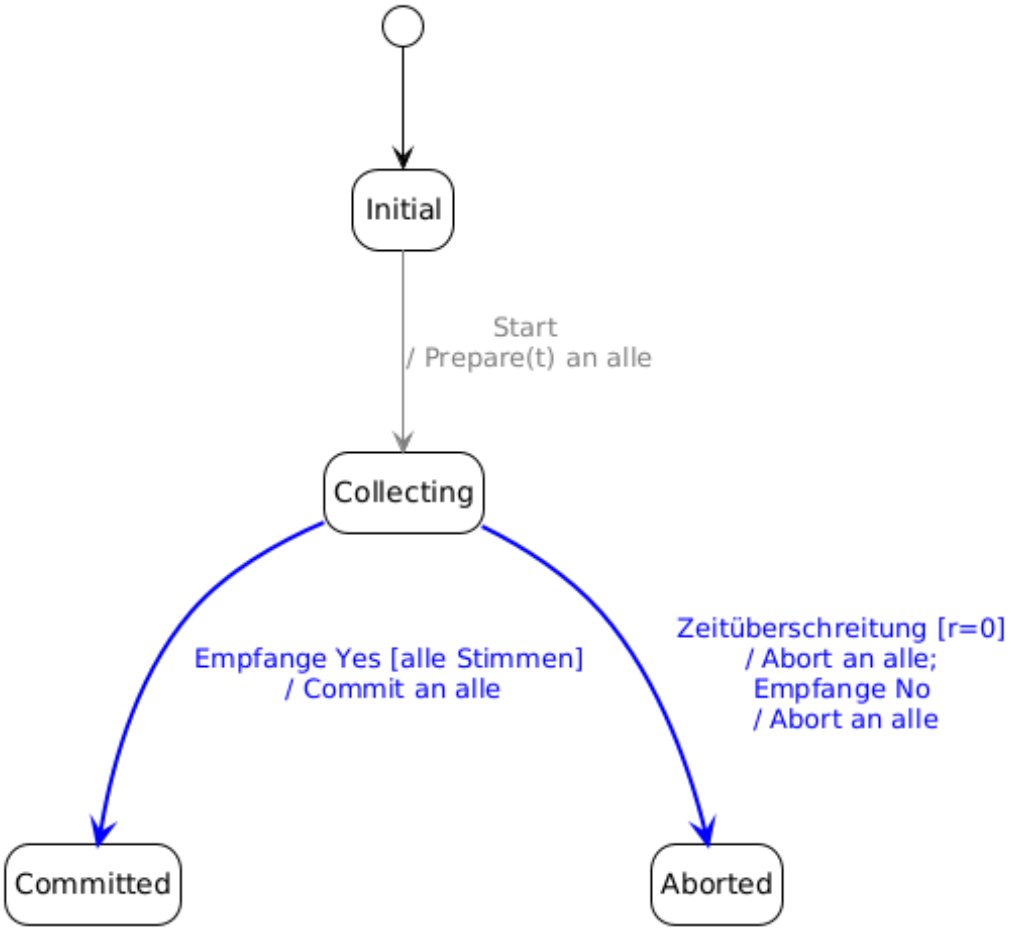
Das Zwei-Phase-Commit-Protokoll
Übergänge des Koordinators



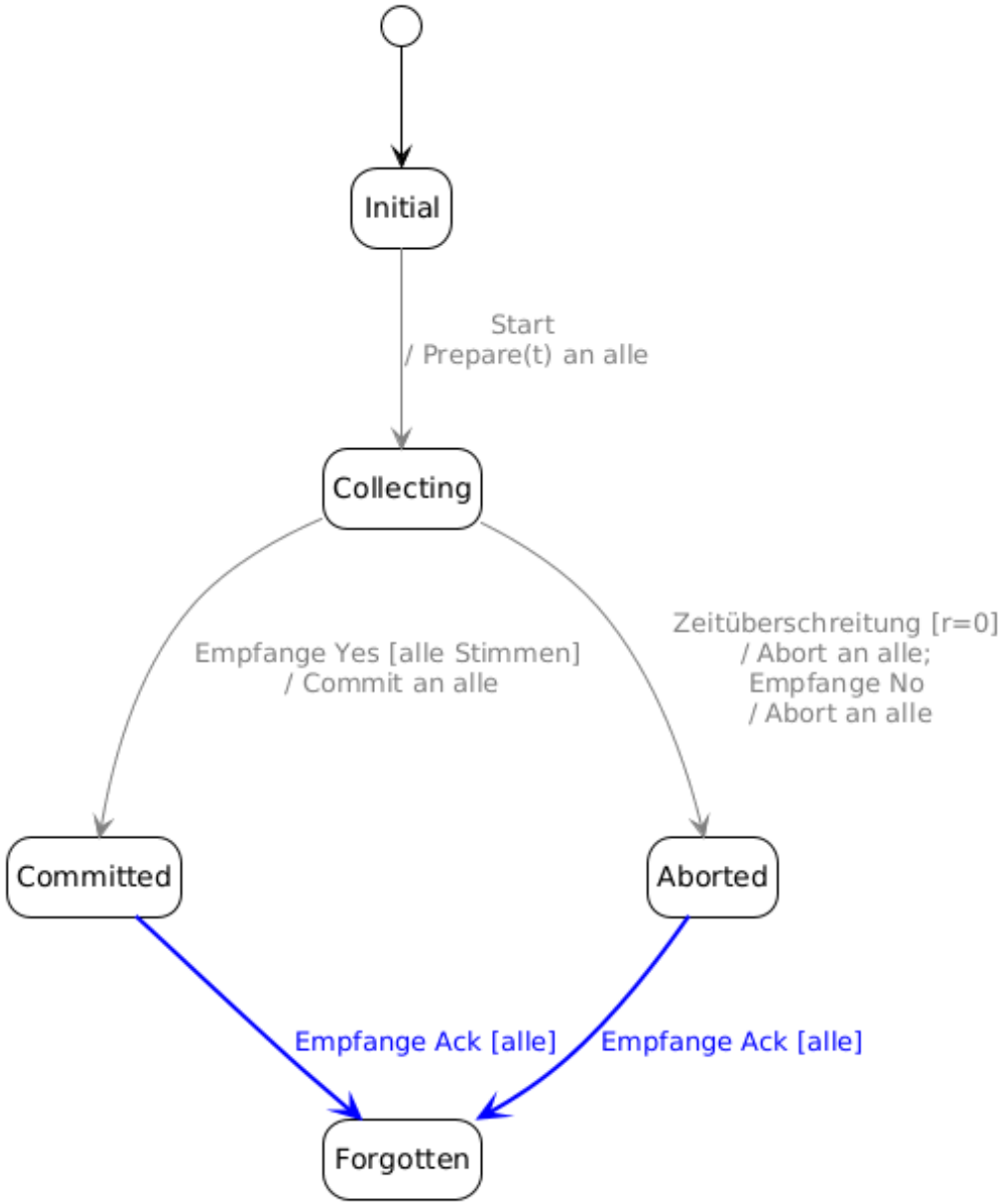
Das Zwei-Phase-Commit-Protokoll
Übergänge des Koordinators



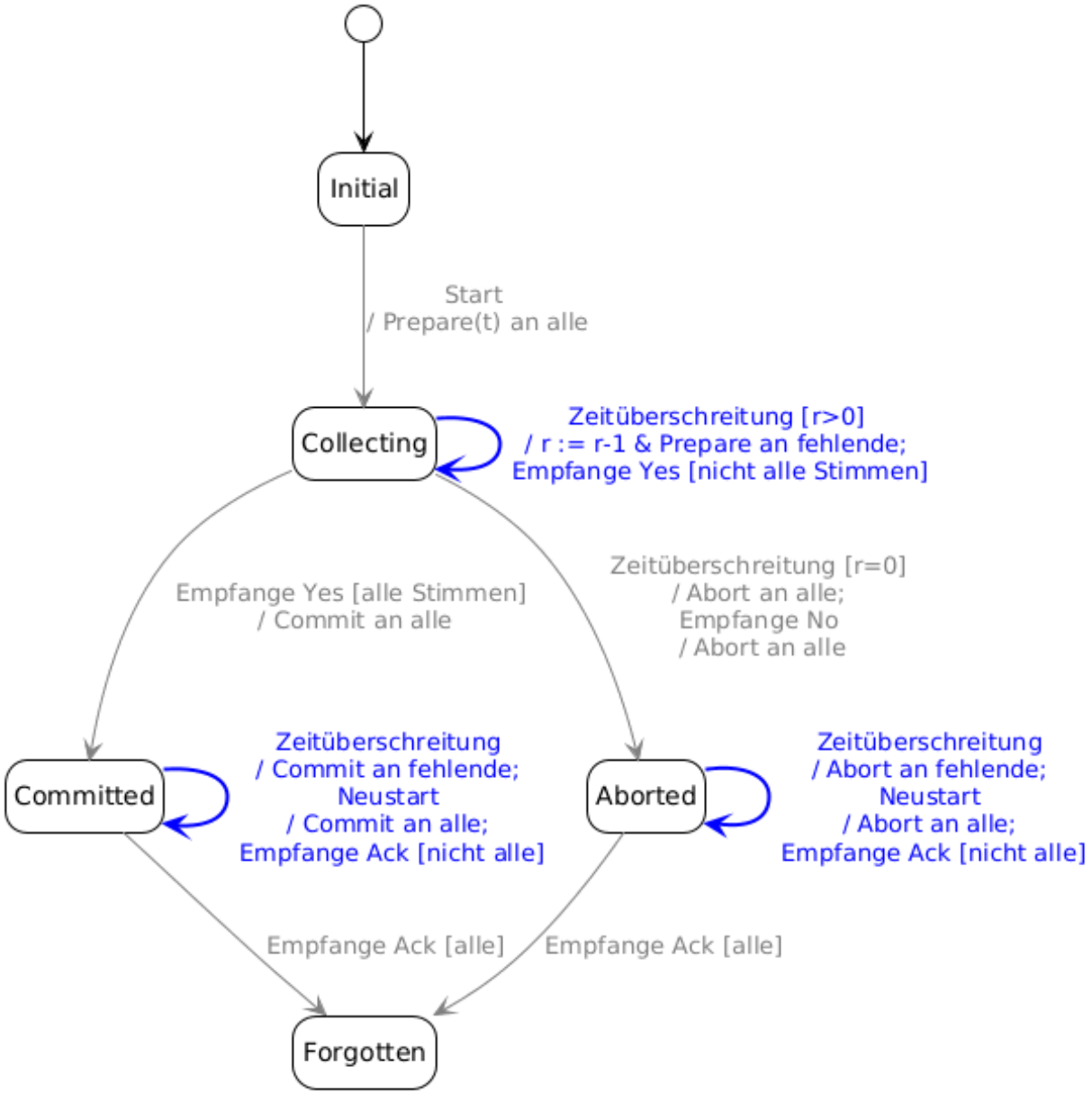
Das Zwei-Phase-Commit-Protokoll
Übergänge des Koordinators



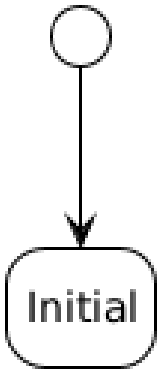
Das Zwei-Phase-Commit-Protokoll
Übergänge des Koordinators



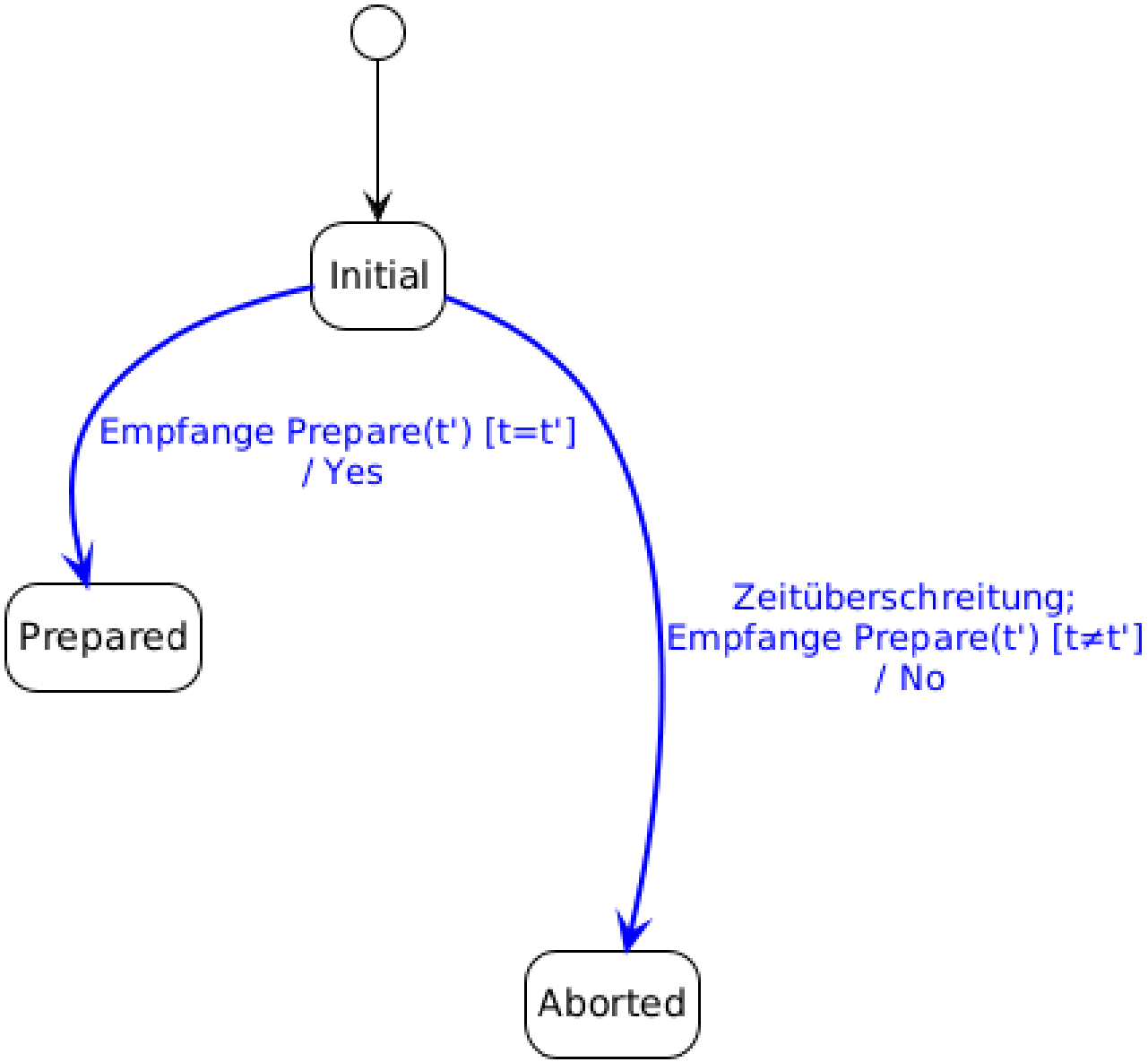
Das Zwei-Phase-Commit-Protokoll
Übergänge des Koordinators



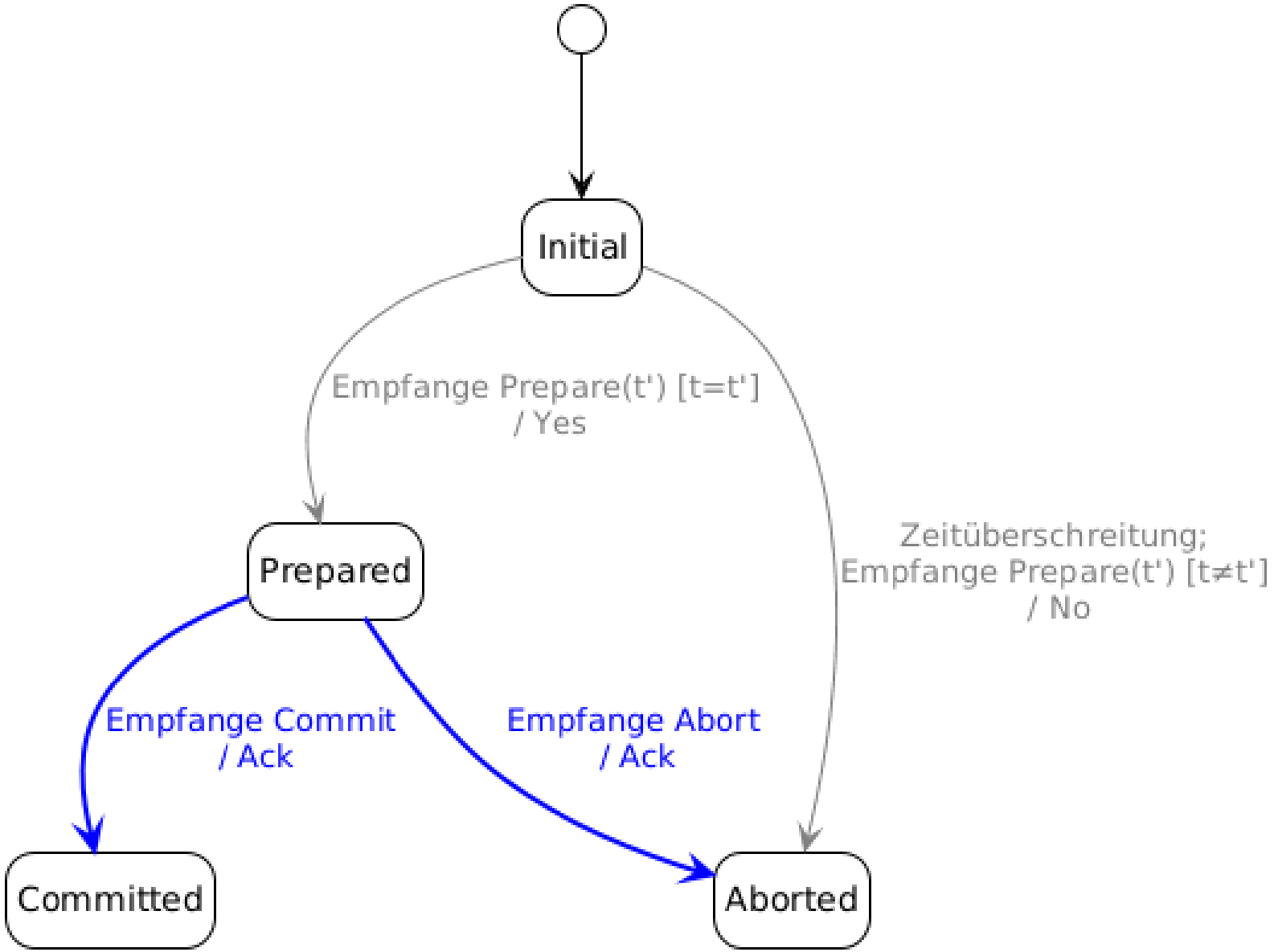
Das Zwei-Phase-Commit-Protokoll
Übergänge eines Teilnehmers



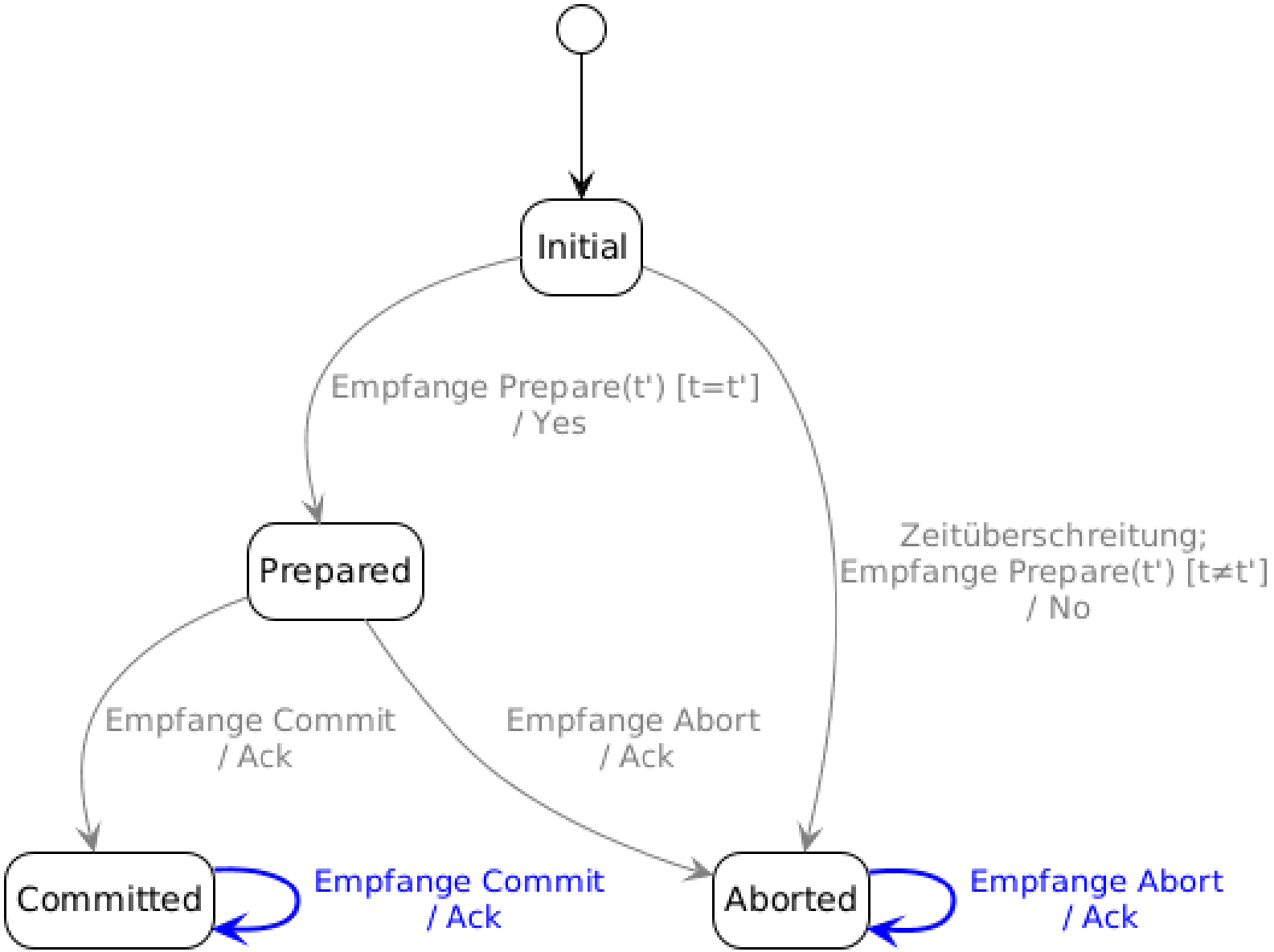
Das Zwei-Phase-Commit-Protokoll
Übergänge eines Teilnehmers



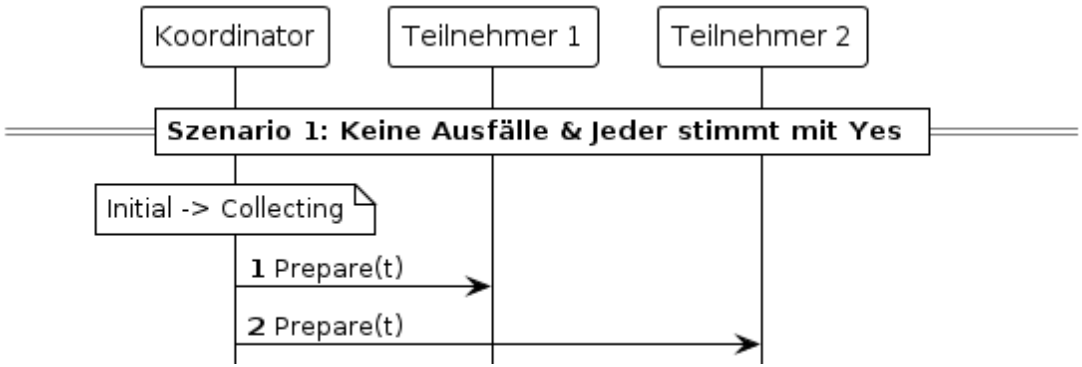
Das Zwei-Phase-Commit-Protokoll
Übergänge eines Teilnehmers



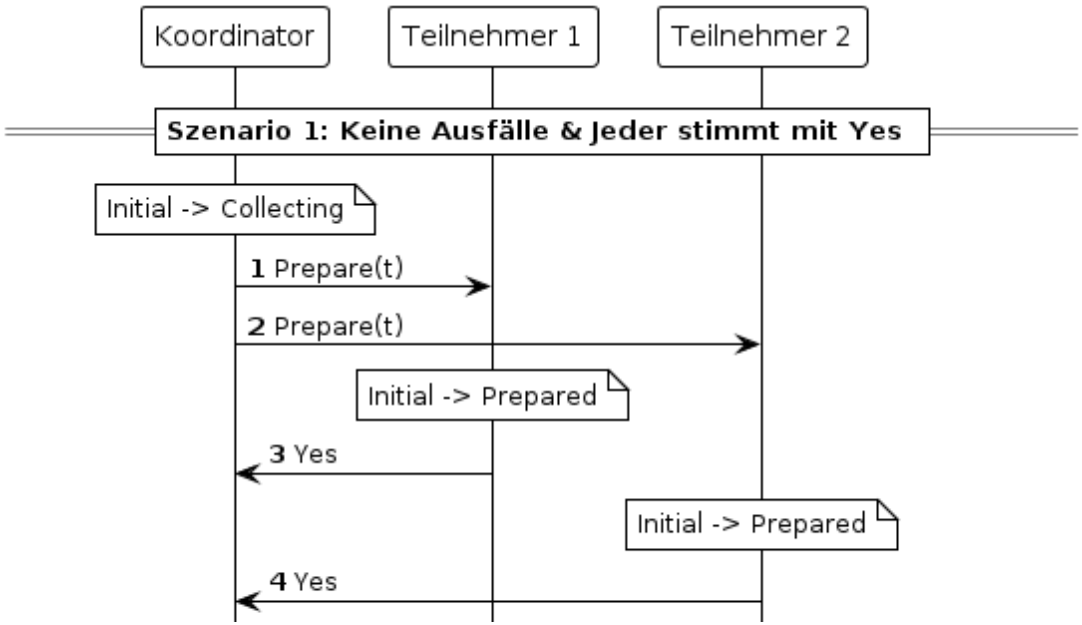
Das Zwei-Phase-Commit-Protokoll
Übergänge eines Teilnehmers



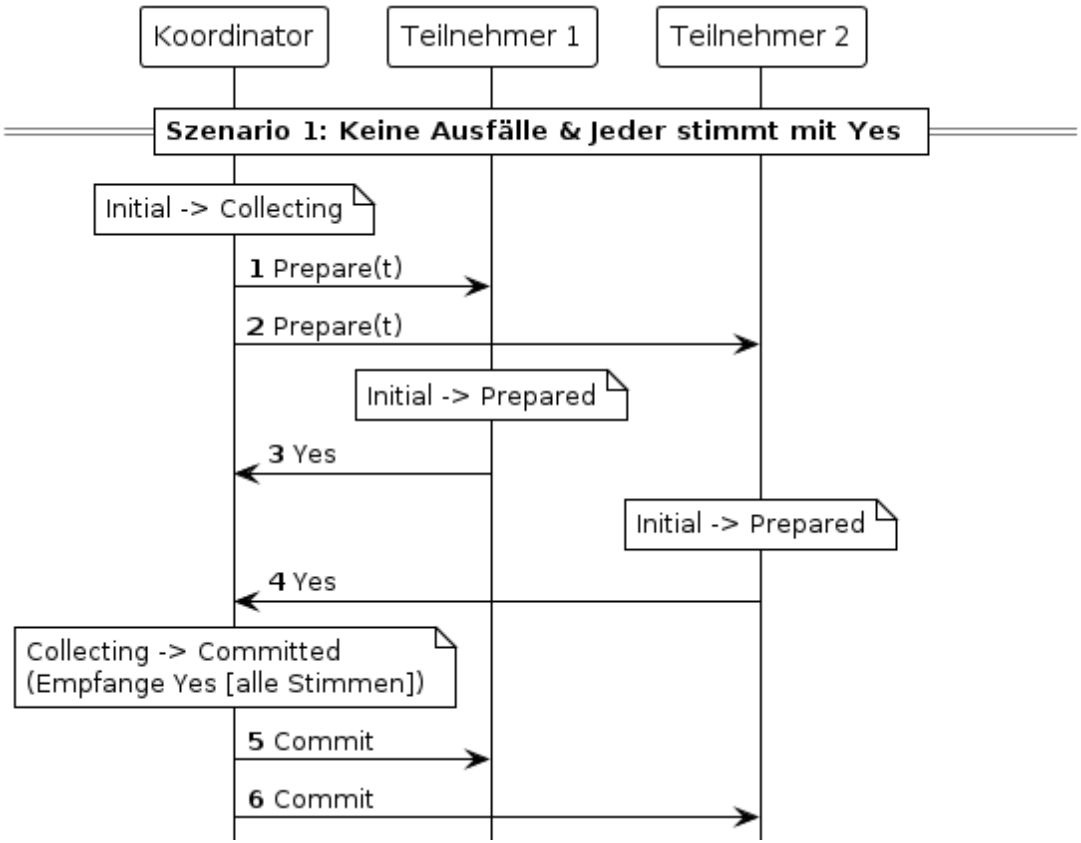
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



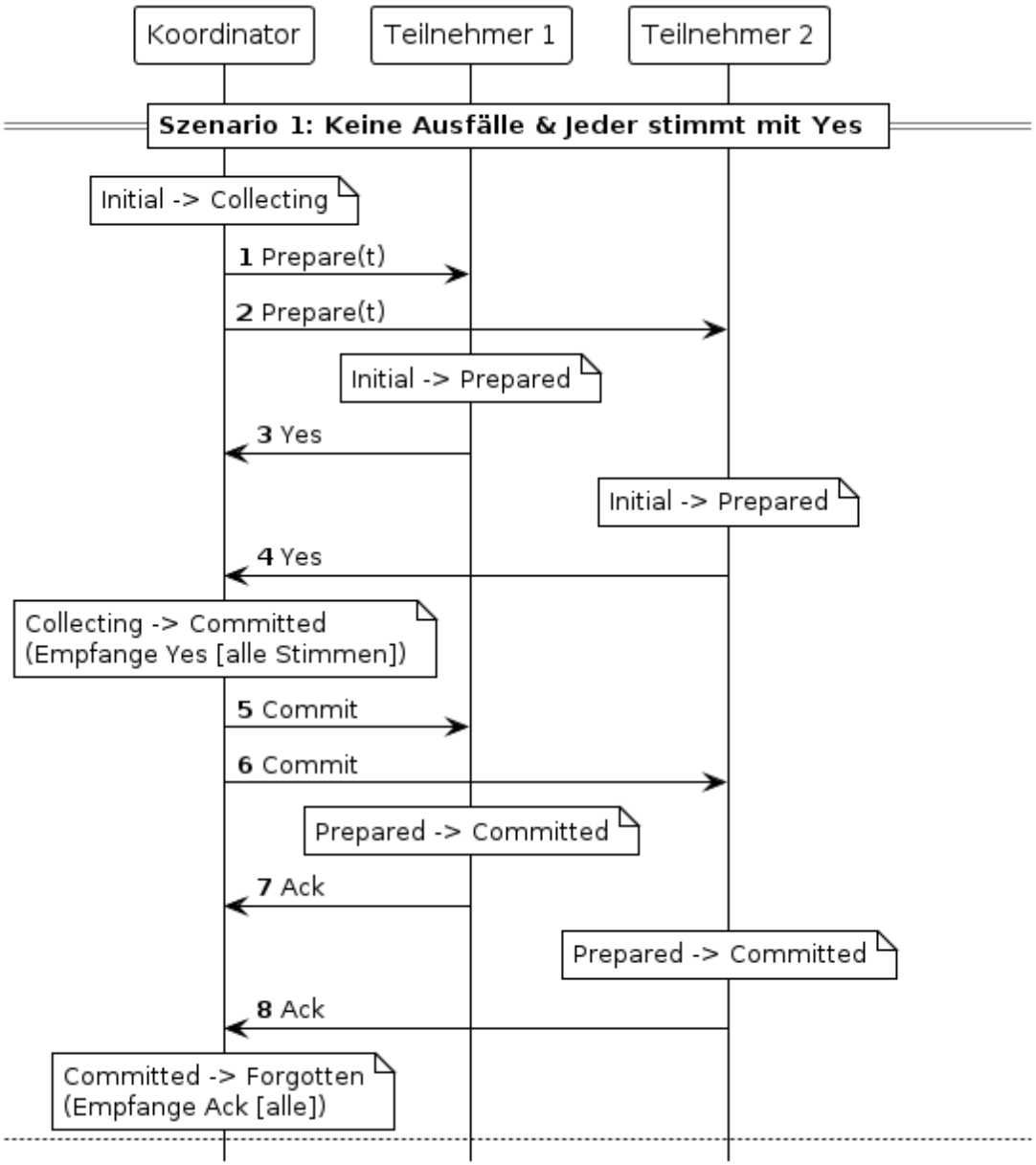
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



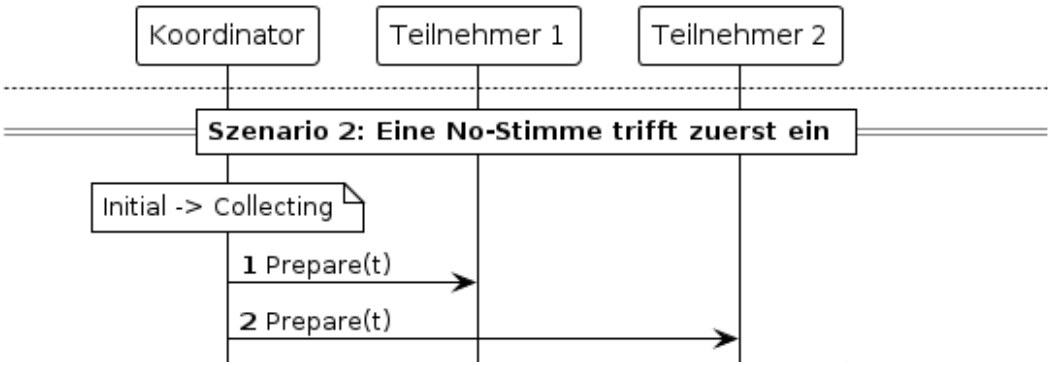
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



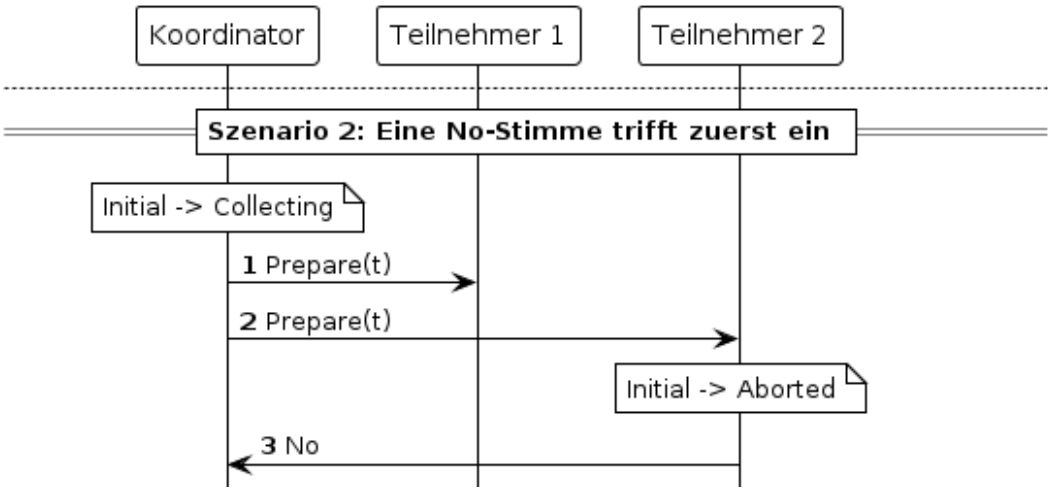
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



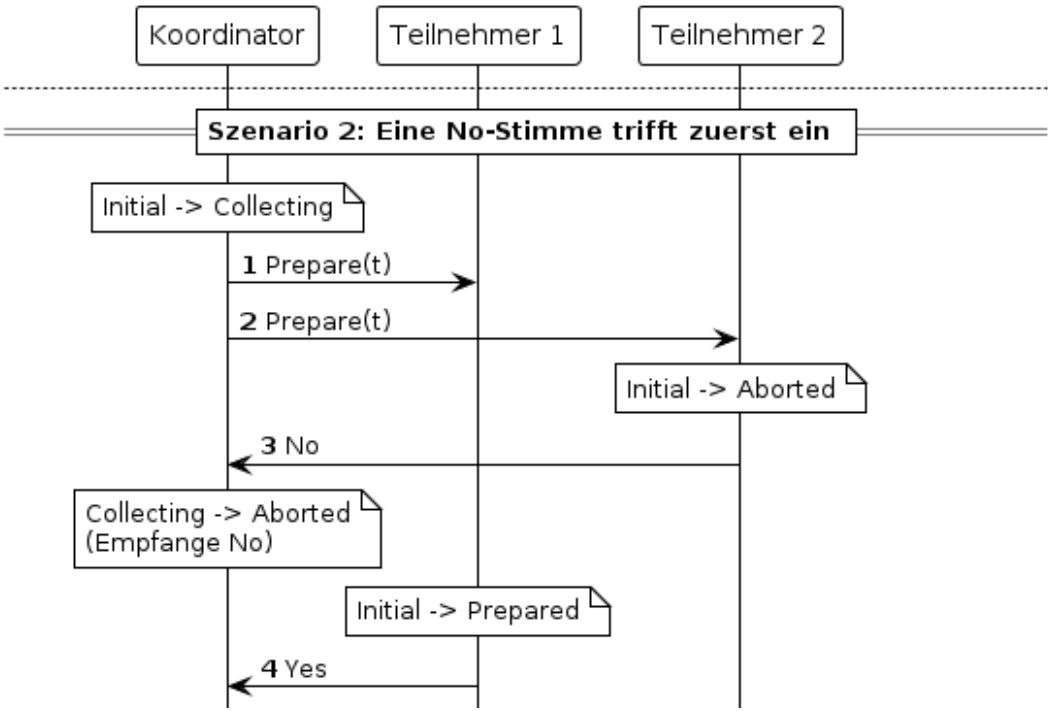
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



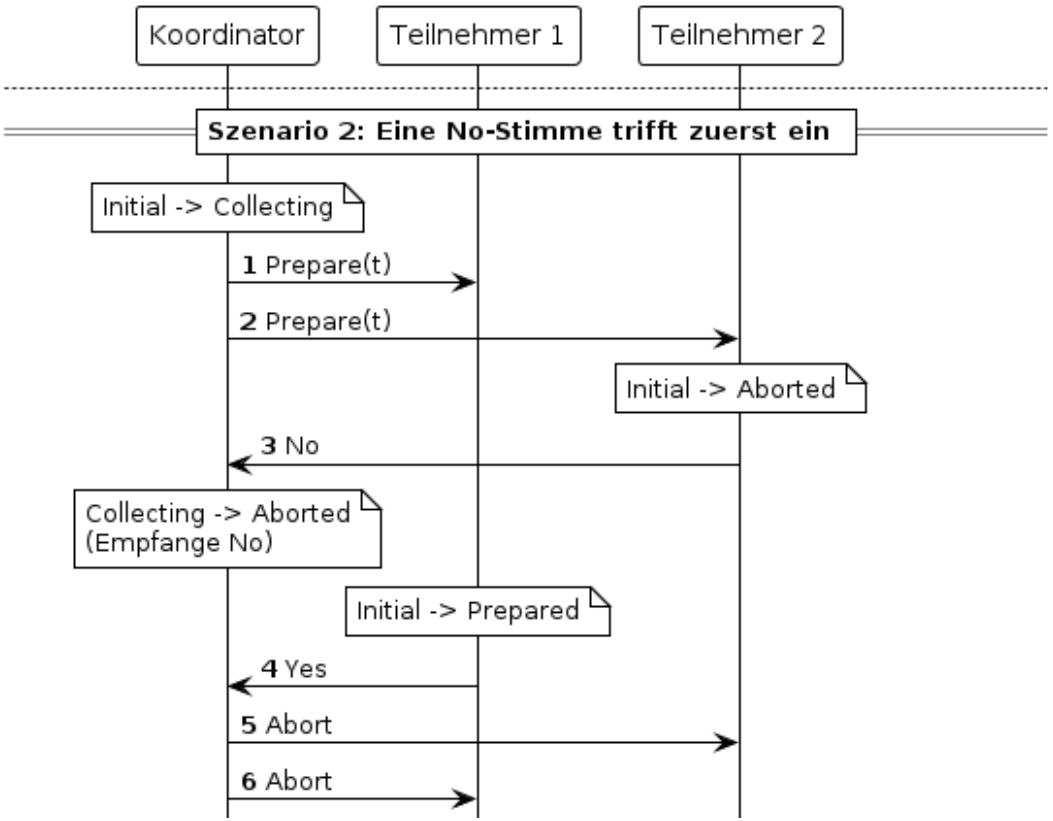
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe

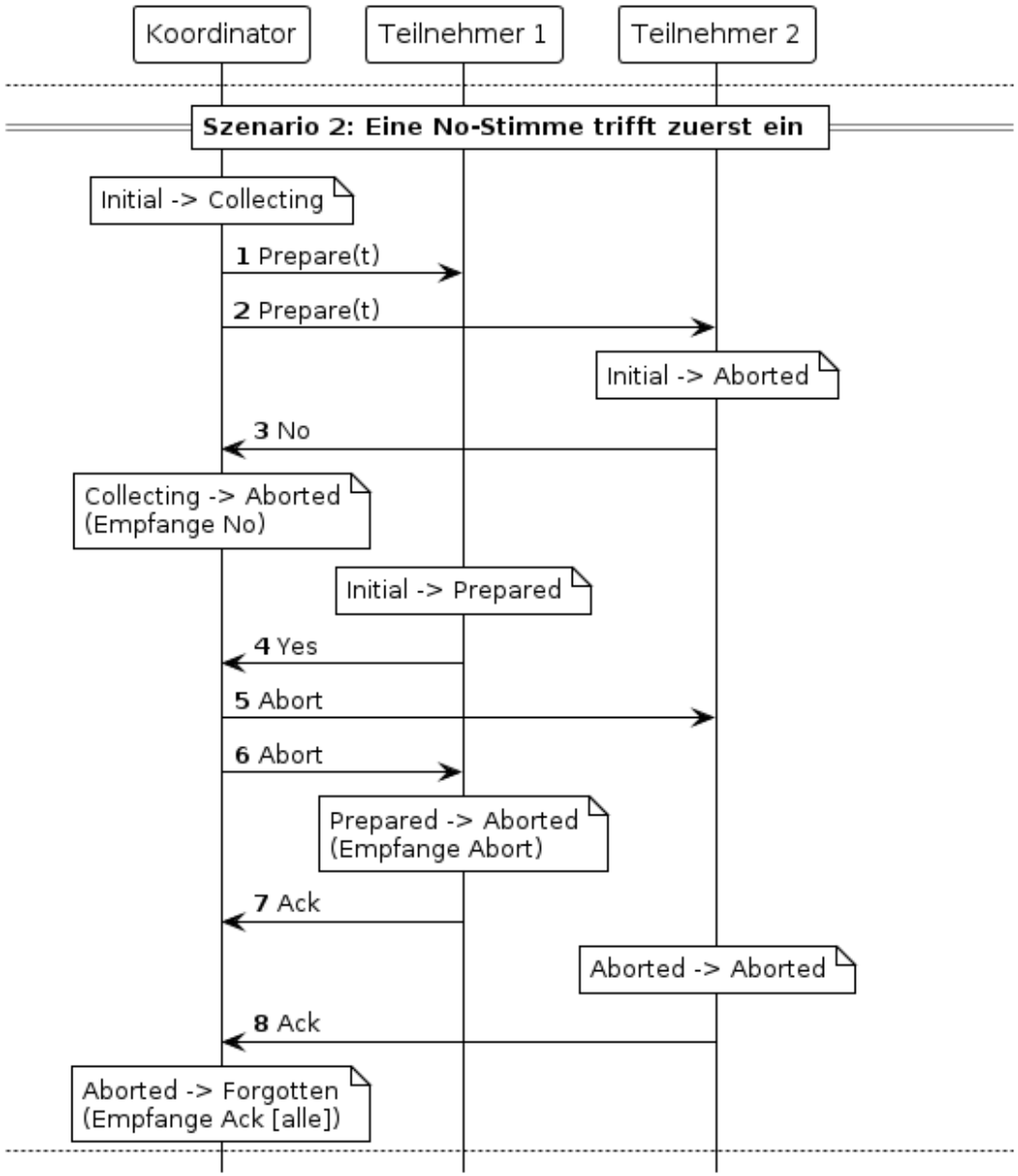


Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe

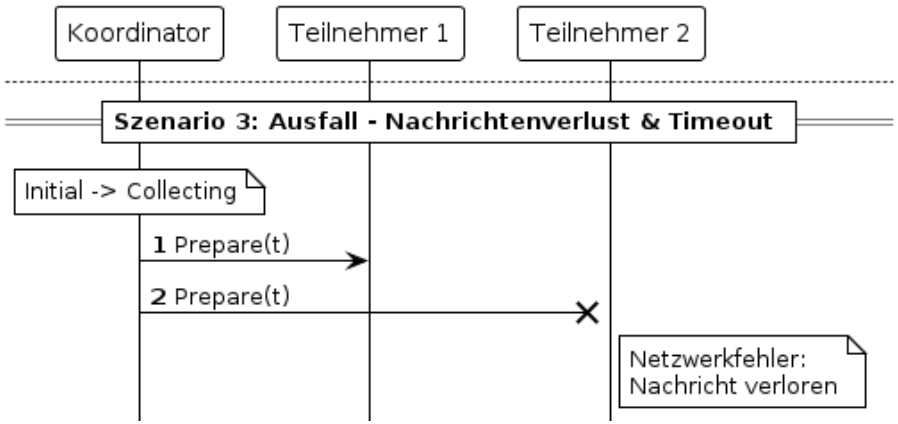


Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe

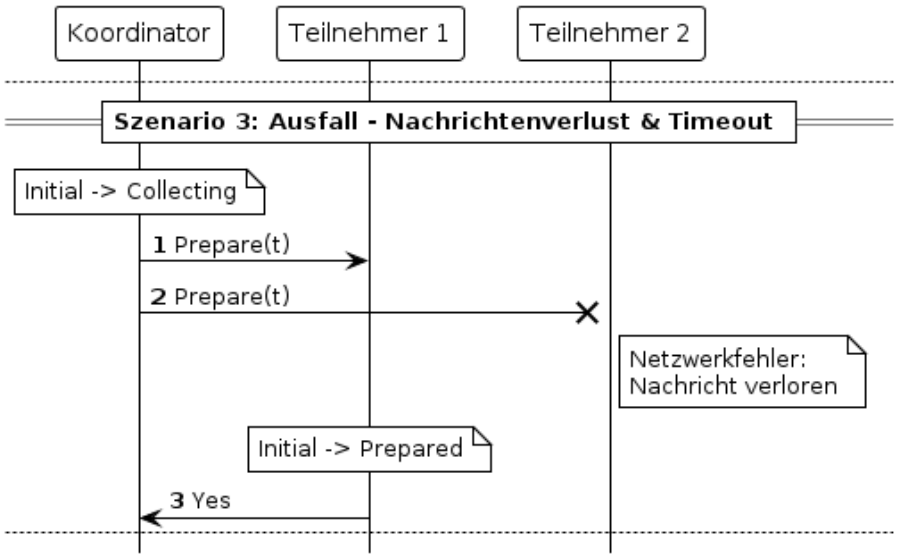



Koordinator
Teilnehmer 1
Teilnehmer 2
Szenario 2: Eine No-Stimme trifft zuerst ein
Initial -> Collecting
1 Prepare(t)
2 Prepare(t)
Initial -> Aborted
3 No
Collecting -> Aborted
(Empfange No)
Initial -> Prepared
4 Yes
5 Abort
6 Abort
Prepared -> Aborted
(Empfange Abort)
7 Ack
Aborted -> Aborted
8 Ack
Aborted -> Forgotten
(Empfange Ack [alle])

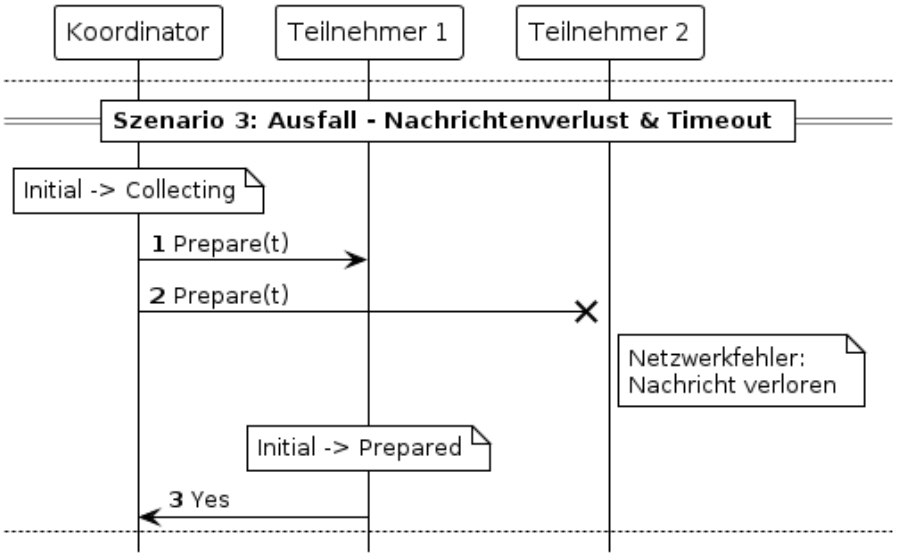
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe

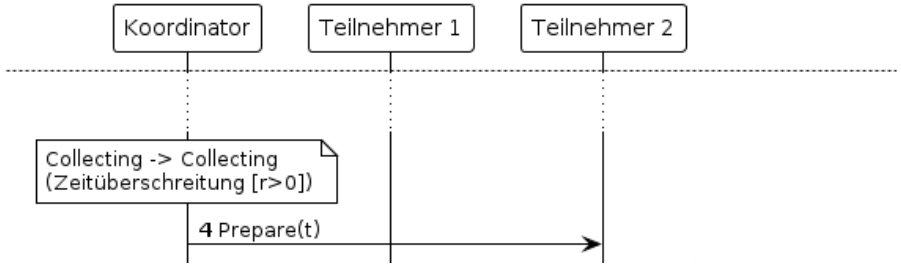


Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe

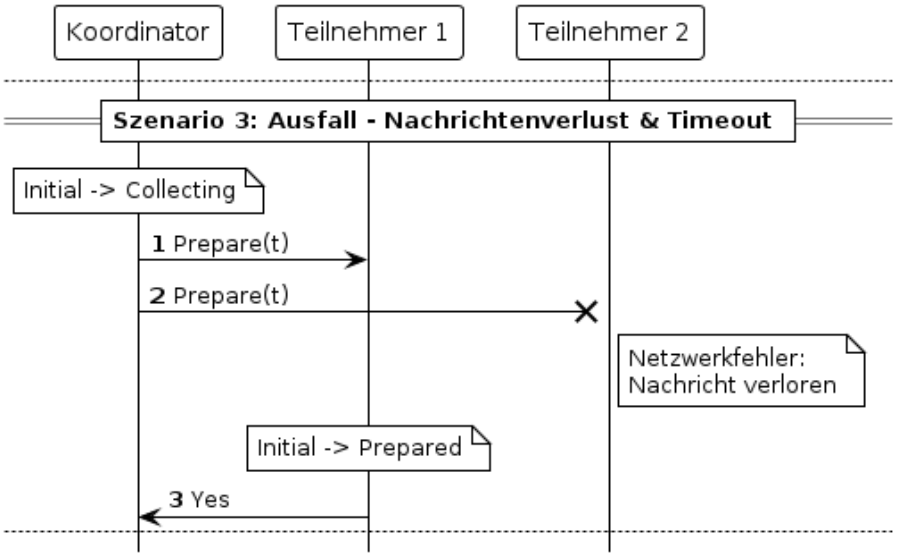


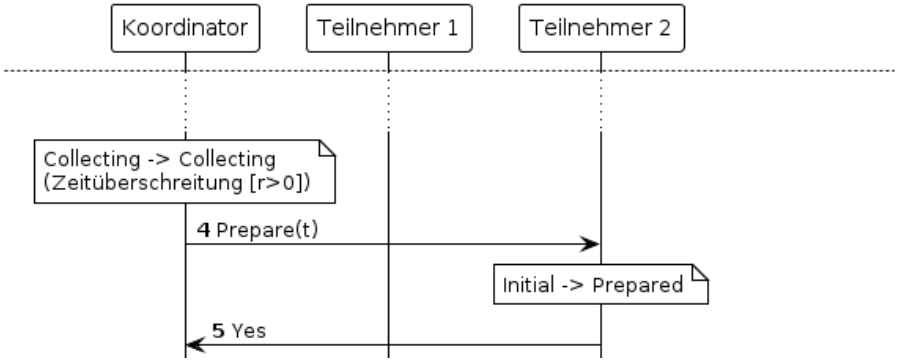
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



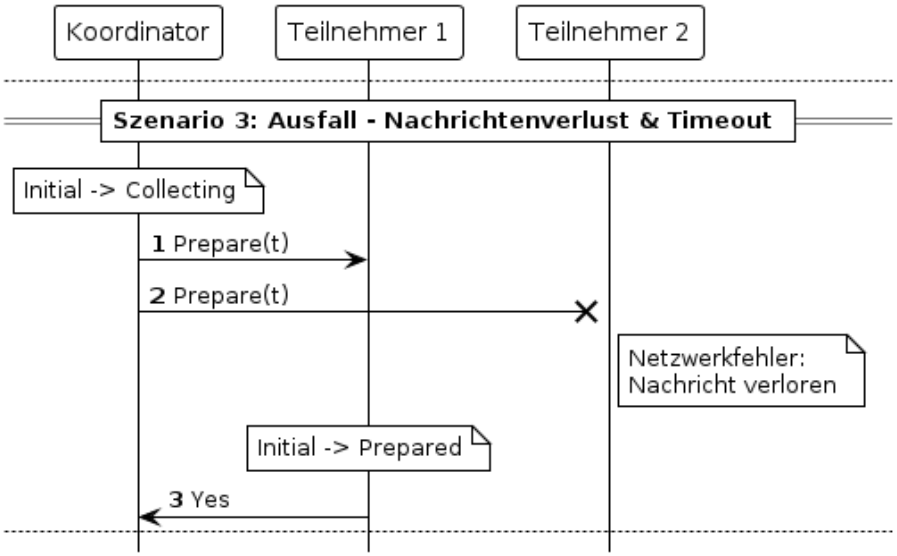


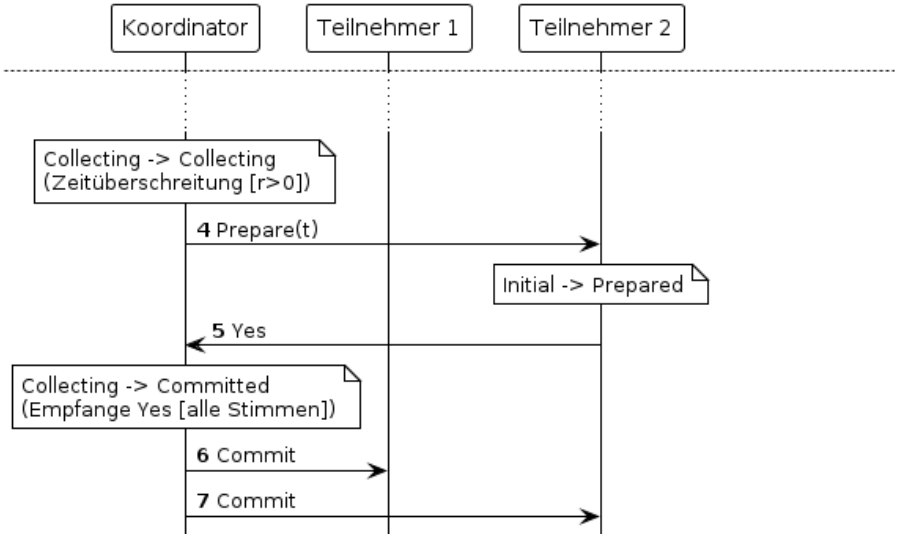
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



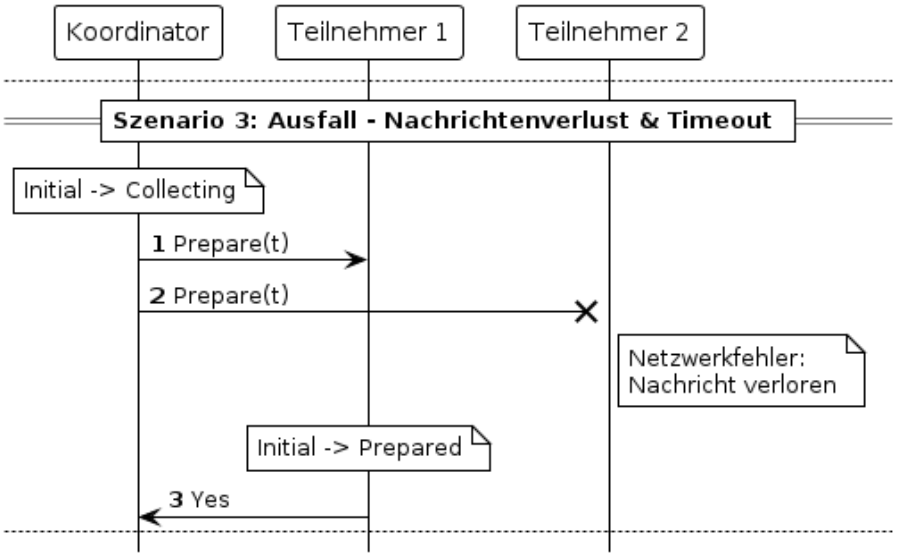


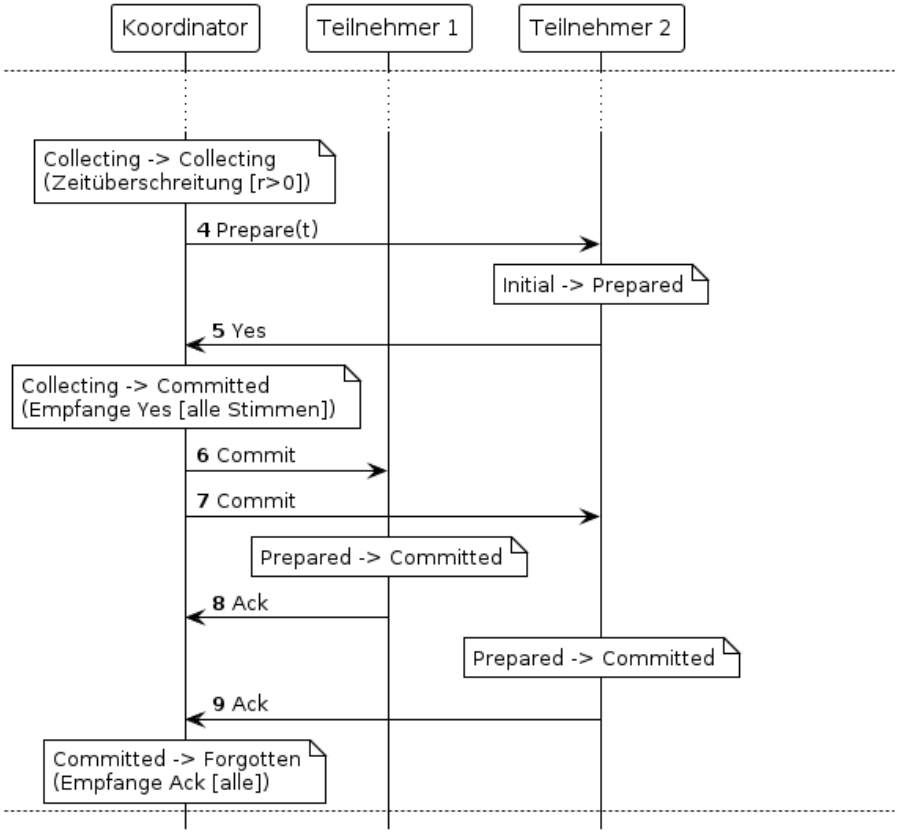
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



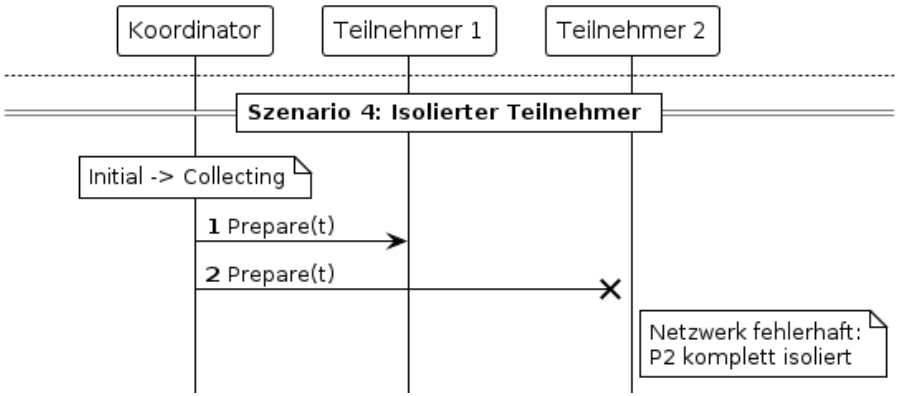


Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe

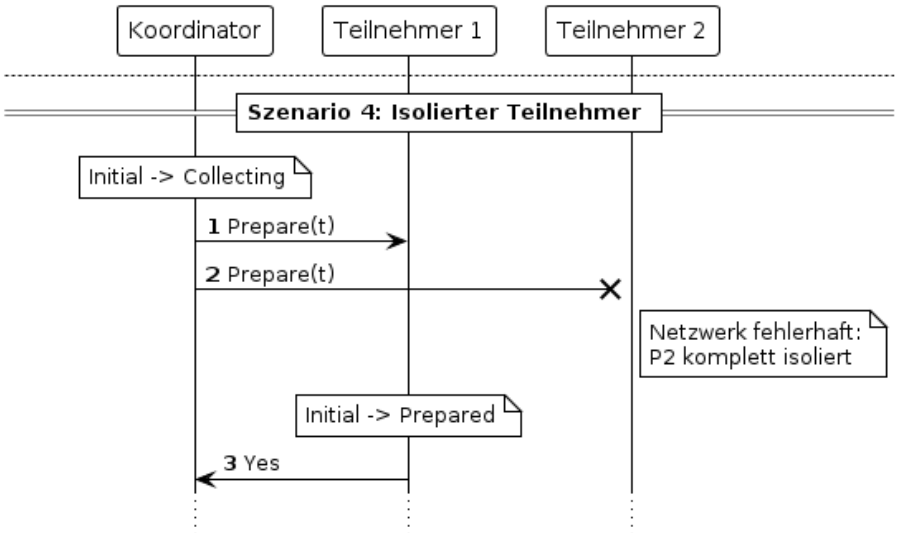




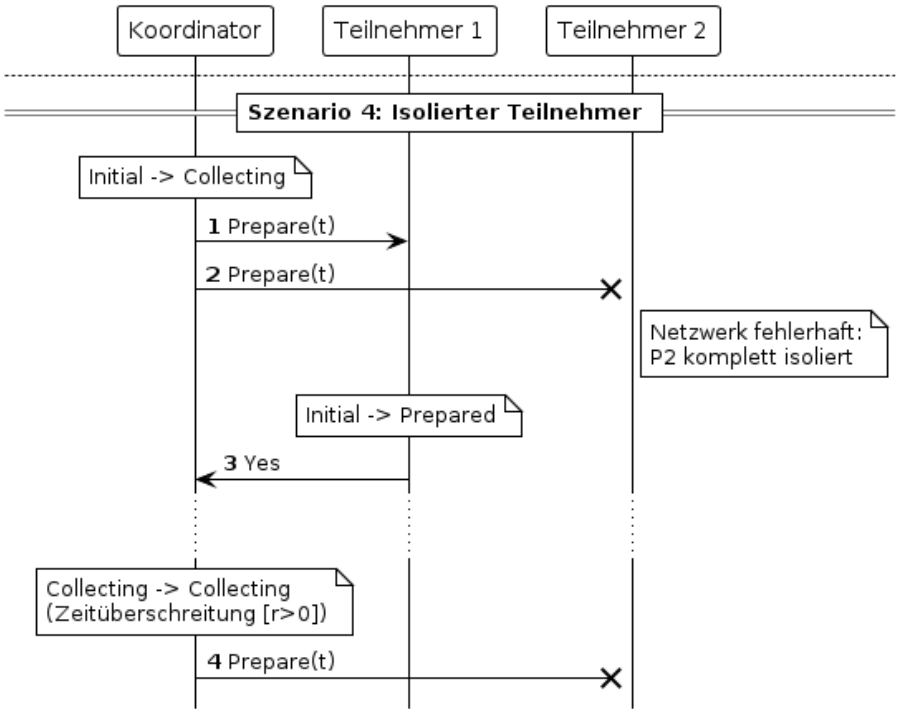
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



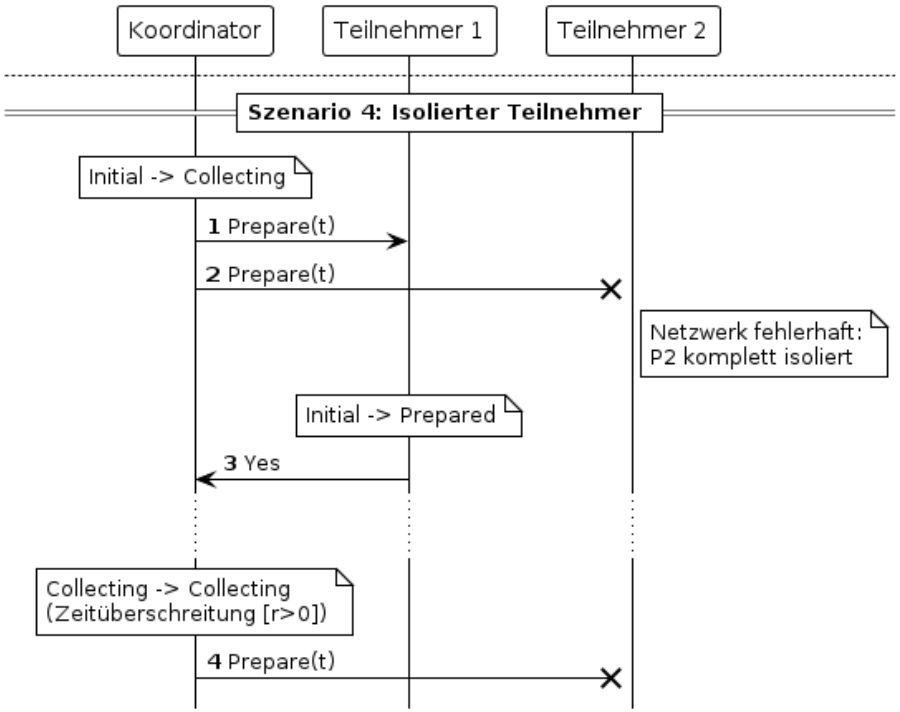
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe

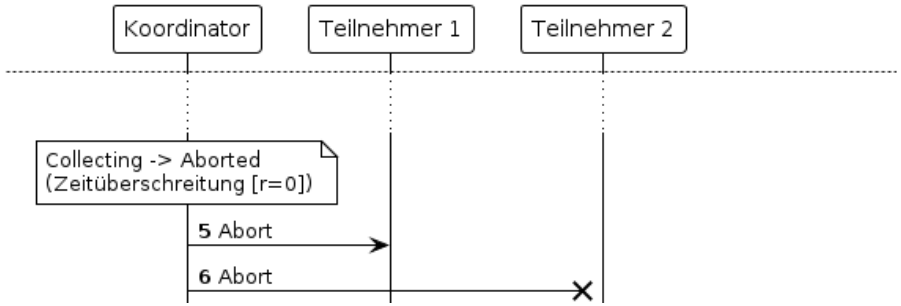


Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe

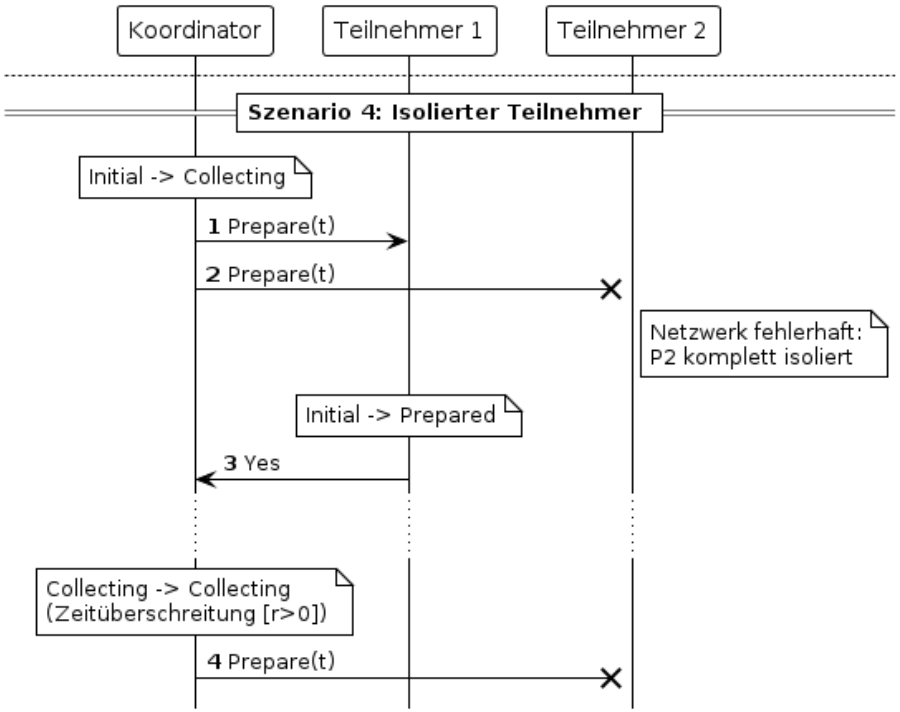


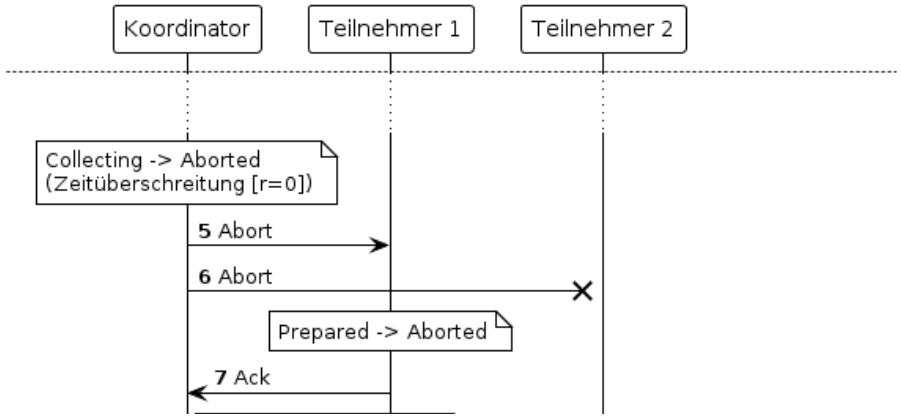
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



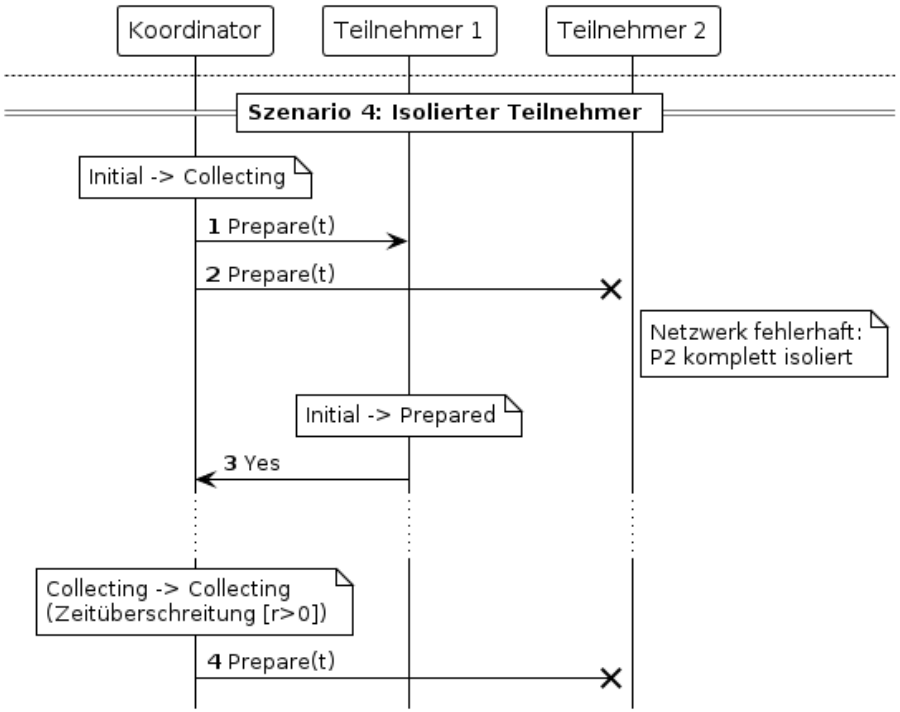


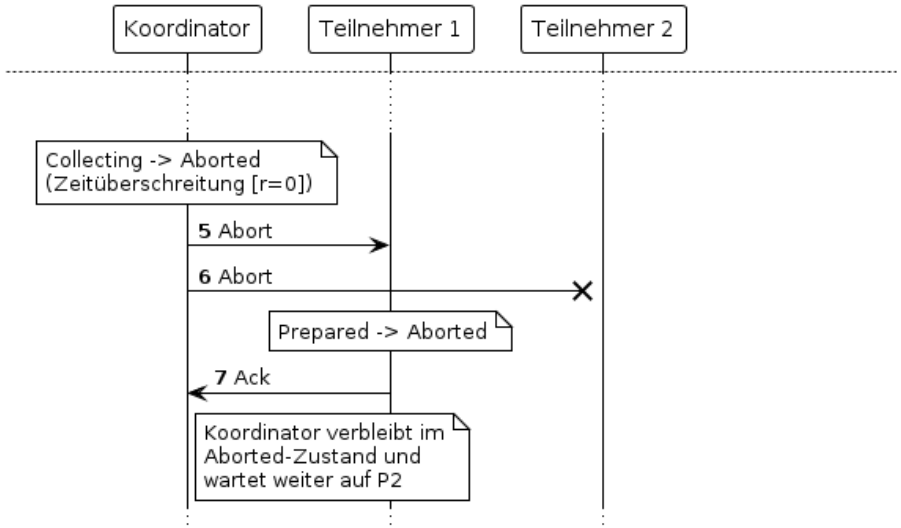
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe



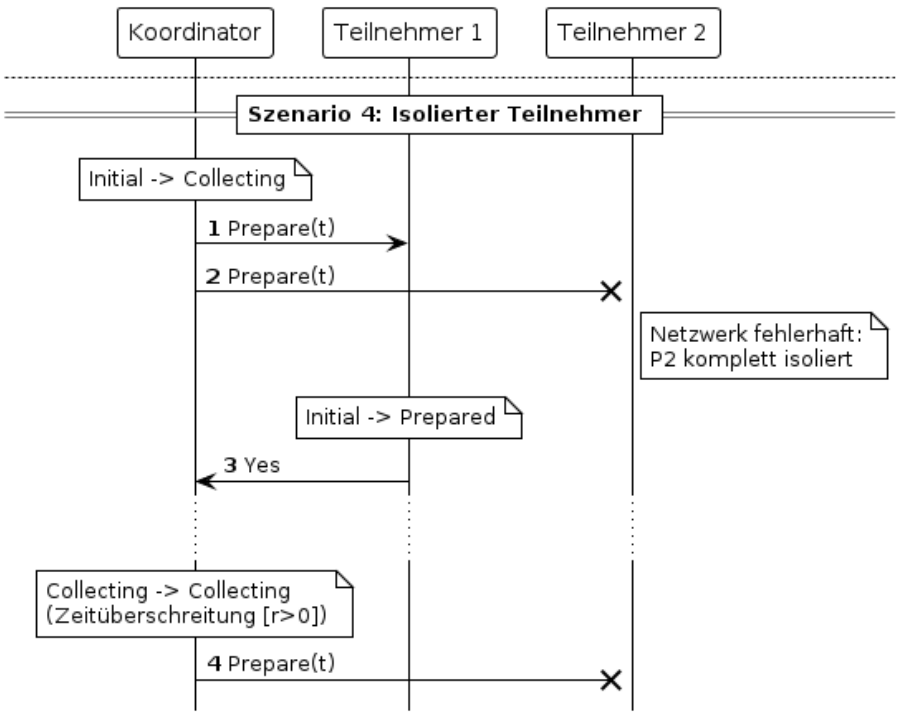


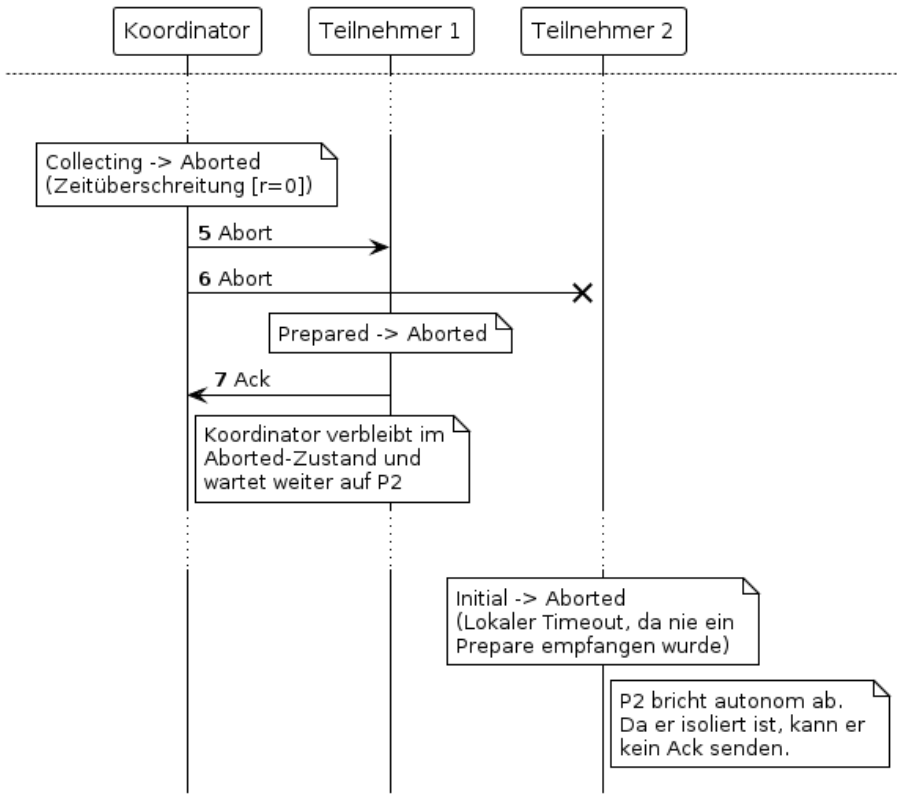
Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe





Das Zwei-Phase-Commit-Protokoll
Beispielhafte Abläufe





Definition einer Signatur S

- *Input-Übergänge* $in(S)$
- *Output-Übergänge* $out(S)$
- *Internal-Übergänge* $int(S)$

Beispielbild!!!!!

Definition eines I/O Automaten A

- *Signatur* $sig(A)$
- *Menge an Zuständen* $states(A)$
- *nichtleere Menge an Startzuständen* $start(A) \subseteq states(A)$
- *Übergangsrelation*

$$trans(A) \subseteq states(A) \times acts(sig(A)) \times states(A)$$

- *Aufgabenpartition* $tasks(A)$

Beispielbild????

Ausführung eines I/O Automaten A

- *Ausführungsfragment* $s_0, \pi_1, s_1, \pi_2, ...$
- $\forall i \geq 0.(s_i, \pi_{i+1}), s_{i+1}) \in trans(A)$
- *Ausführung falls* $s_0 \in starts(A)$

Beispielbild????

Komposition zweier I/O Automaten A & B

- *Automaten müssen kompatibel sein*
- *Signaturen werden Verknüpft*
- *entstehende Zustände sind Vektoren* (s_a, s_b) *mit* $s_i \in start(I)$
- A *und* B *dürfen bei gemeinsamen Übergängen gleichzeitig agieren*

OUTPUTS bleiben OUTPUTS für weitere Komposition!

```
datatype 't msg = Prepare (transaction: 't) | Yes | No |
  ↪  Commit | Abort | Ack
datatype ('proc, 'msg) event
  = Start
  | Receive (msg_sender: 'proc) (recv_msg: 'msg)
  | Timeout
  | Restart
```

```
datatype 't msg = Prepare (transaction: 't) | Yes | No |
  ↪  Commit | Abort | Ack
datatype ('proc, 'msg) action =
    Start "'proc"
  | Send "'proc" "'proc" "'msg"
  | Receive "'proc" "'proc" "'msg"
  | Timeout "'proc"
  | Restart "'proc"
```

```
50  definition coord_asig :: "'proc ⇒ ('proc,'t) action
     ↪  signature" where
51  "coord_asig pid =
52   ({Receive q pid m | q m. True} ∪ {Start pid, Timeout pid,
      ↪  Restart pid},
53    {Send pid q m | q m. True},
54    {})"
89  definition parts_asig :: "'proc set ⇒ ('proc,'t msg) action
     ↪  signature" where
90  "parts_asig P =
91   ({Receive q p m | q p m. p ∈ P} ∪ {Timeout p | p. p ∈ P} ∪
      ↪  {Restart p | p. p ∈ P},
92    {Send p q m | p q m. p ∈ P},
93    {})"
113 definition chan_asig :: "('proc,'t) action signature" where
114 "chan_asig =
115  ({Send s r m | s r m. True},
116   {Receive s r m | s r m. True},
117   {})"
```

```
datatype ('t, 'proc) state = Initial (transaction: 't) (all: "'proc set") (retry_count:"nat")
  | Collecting (transaction: 't) (all: "'proc set") (yes:"'proc set") (retry_count:"nat")
  | Prepared
  | Committed (all: "'proc set") (ack: "'proc set")
  | Aborted (all: "'proc set") (ack: "'proc set")
  | Forgotten
```

```
datatype ('t,'proc) cstate = CInitial (transaction: 't) (all: "'proc set") (retry_count:"nat")
  | Collecting (transaction: 't) (all: "'proc set") (yes:"'proc set") (retry_count:"nat")
  | CCommitted (all: "'proc set") (ack: "'proc set")
  | CAborted (all: "'proc set") (ack: "'proc set")
  | Forgotten
datatype ('t,'proc) coord_state = CState "('t,'proc) cstate" "('proc × 'proc × 't msg) set"
```

```
7  datatype ('t, 'proc) state = Initial (transaction: 't) (all:
   ↪  "'proc set") (retry_count:"nat")
8    | Collecting (transaction: 't) (all: "'proc set")
   ↪  (yes:"'proc set") (retry_count:"nat")
9    | Prepared
10   | Committed (all: "'proc set") (ack: "'proc set")
11   | Aborted (all: "'proc set") (ack: "'proc set")
12   | Forgotten
```

```
61 datatype 't pstate = PInitial (transaction: 't)
62   | Prepared
63   | PCommitted
64   | PAborted
65 datatype ('t,'proc) part_state = PState "'t pstate" "('proc
   ↪  × 'proc × 't msg) set"
```

```
7   datatype ('t, 'proc) state = Initial (transaction: 't) (all:
    ↪  "'proc set") (retry_count:"nat")
8     | Collecting (transaction: 't) (all: "'proc set")
      ↪  (yes:"'proc set") (retry_count:"nat")
9     | Prepared
10    | Committed (all: "'proc set") (ack: "'proc set")
11    | Aborted (all: "'proc set") (ack: "'proc set")
12    | Forgotten
```

```
106 type_synonym ('proc,'t) chan_state = "('proc × 'proc × 't
    ↪  msg) set"
```

```
14  fun coordinator_step::
15      "'proc ⇒ ('t, 'proc) state ⇒ ('proc, 't msg) event ⇒
        ↪  ('t, 'proc) state × ('proc, 't msg) send set"  where
34  fun participant_step::
35      "('t, 'proc) state ⇒ ('proc, 't msg) event ⇒ ('t,
        ↪  'proc) state × ('proc, 't msg) send set" where
```

```
24  fun coordinator_step::
25      "'proc ⇒ ('t, 'proc) cstate ⇒ ('proc, 't msg) action ⇒
        ↪  ('t, 'proc) cstate × ('proc × 't msg) set"  where
67  fun participant_step::
68      "'proc ⇒ 't pstate ⇒ ('proc, 't msg) action ⇒ 't pstate
        ↪  × ('proc × 't msg) set"  where
```

```
16  ‹coordinator_step pid (Initial t a r) Start = (Collecting t
    ↪  a {pid} r, {Send p (Prepare t) | p. p ∈ a})› |
```

```
26  ‹coordinator_step pid (CInitial t a r) (Start _) =
    ↪  (Collecting t a {pid} r, {(q, (Prepare t)) | q. q ∈ a -
    ↪  {pid}})› |
```

```
17    ‹coordinator_step pid (Collecting t a y r) (Receive sender
      ↪  msg) =
18      (case msg of
19        Yes ⇒(if y ∪ {sender} = a then (Committed a {pid},
          ↪  {Send p (Commit) | p. p ∈ a}) else (Collecting t
          ↪  a (y ∪ {sender}) r, {})) |
20        No ⇒(Aborted a {pid}, {Send p (Abort) | p. p ∈ a}) |
21        _ ⇒(Collecting t a y r, {}))› |
```

```
27    ‹coordinator_step pid (Collecting t a y r) (Receive sender
      ↪  _ msg) =
28      (case msg of
29        Yes ⇒(if y ∪ {sender} = a then (CCommitted a {pid},
          ↪  {(q, Commit) | q. q ∈ a - {pid}}) else
          ↪  (Collecting t a (y ∪ {sender}) r, {})) |
30        No ⇒(CAborted a {pid}, {(q, Abort) | q. q ∈ a -
          ↪  {pid}}) |
31        _ ⇒(Collecting t a y r, {}))› |
```

```
22    ‹coordinator_step pid (Collecting t a y 0) Timeout =
      ↪  (Aborted a {pid}, {Send p (Abort) | p. p ∈ a})› |
23    ‹coordinator_step _ (Collecting t a y (Suc r)) Timeout =
      ↪  (Collecting t a y r, {Send p (Prepare t) | p. p ∈ (a -
      ↪  y)})› |
```

```
32    ‹coordinator_step pid (Collecting t a y 0) (Timeout _) =
      ↪  (CAborted a {pid}, {(q, Abort) | q. q ∈ a - {pid}})› |
33    ‹coordinator_step pid (Collecting t a y (Suc r)) (Timeout
      ↪  _) = (Collecting t a y r, {(q, Prepare t) | q. q ∈ (a -
      ↪  y)})› |
```

```
26    ‹coordinator_step _ (Committed a ack') Timeout = (Committed
      ↪  a ack', {Send p Commit | p. p ∈ (a - ack')})› |
27    ‹coordinator_step _ (Committed a ack') Restart = (Committed
      ↪  a ack', {Send p Commit | p. p ∈ a})› |
```

```
36    ‹coordinator_step pid (CCommitted a ack') (Timeout _) =
      ↪  (CCommitted a ack', {(q, Commit) | q. q ∈ (a - ack')})›
      ↪  |
37    ‹coordinator_step pid (CCommitted a ack') (Restart _) =
      ↪  (CCommitted a ack', {(q, Commit) | q. q ∈ a - {pid}})›
      ↪  |
```

```
44  definition coord_trans ::
45    "'proc ⇒ (('t,'proc) coord_state × ('proc,'t msg) action ×
      ↪  ('t,'proc) coord_state) set" where
46  "coord_trans pid =
47    {(CState s msgs, a, CState s' (msgs ∪ ((λmsg. (pid, msg)) `
      ↪  sent))) | s a s' msgs sent. coordinator_step pid s a =
      ↪  (s', sent) ∧ (∃s m. a = Start pid ∨ a = Receive s pid m
      ↪  ∨ a = Timeout pid ∨ a = Restart pid)}
48    ∪ {(CState s msgs, Send pid rcpt msg, CState s (msgs -
      ↪  {(pid,rcpt,msg)})) | s msgs rcpt msg. (pid,rcpt,msg) ∈
      ↪  msgs}"

56  definition Coordinator :: "'p ⇒ 't ⇒ 'p set ⇒ nat ⇒
    ↪  (('p,'t msg) action, ('t,'p) coord_state) ioa" where
57  "Coordinator pid t a r = (coord_asig pid, {CState (CInitial t
    ↪  a r) {}}, coord_trans pid, {}, {})"
```

```
81  definition part_trans ::
82    "'proc ⇒ (('t,'proc) part_state × ('proc,'t msg) action ×
      ↪  ('t,'proc) part_state) set" where
83  "part_trans pid =
84    {(PState s msgs, a, PState s' (msgs ∪ ((λmsg. (pid, msg)) `
      ↪  sent))) | s a s' msgs sent. participant_step pid s a =
      ↪  (s', sent) ∧ (∃s m. a = Receive s pid m ∨ a = Timeout
      ↪  pid ∨ a = Restart pid)}
85    ∪ {(PState s msgs, Send pid rcpt msg, PState s (msgs -
      ↪  {(pid,rcpt,msg)})) | s msgs rcpt msg. (pid,rcpt,msg) ∈
      ↪  msgs}"
```

```
95  definition parts_trans :: "'proc set ⇒ (('proc ⇒ ('t,'proc)
    ↪  part_state) × ('proc,'t msg) action × ('proc ⇒
    ↪  ('t,'proc) part_state)) set" where
96  "parts_trans P = { (s, a, s').
97      ∀p. if p ∈ P ∧ ((∃q m. a = Receive q p m) ∨ a = Timeout p
        ↪  ∨ a = Restart p ∨ (∃q m. a = Send p q m))
98          then (s p, a, s' p) ∈ part_trans p
99          else s' p = s p }"
101 definition Participants :: "'proc set ⇒ ('proc ⇒ 't) ⇒
    ↪  (('proc,'t msg) action, 'proc ⇒ ('t,'proc) part_state)
    ↪  ioa" where
102 "Participants P t = (parts_asig P, {λp. PState (PInitial (t
    ↪  p)) {}}, parts_trans P, {}, {})"
```

```
108 definition chan_trans :: "(('proc,'t) chan_state × ('proc,'t msg) action × ('proc,'t) chan_state) set" where
109 "chan_trans =
110   {(M, Send s r m, M ∪ {(s,r,m)}) | M s r m. True}
111   ∪ {(M, Receive s r m, M) | M s r m. (s,r,m) ∈ M}"
119 definition Channel :: "(('proc,'t msg) action, ('proc,'t) chan_state) ioa" where
120 "Channel = (chan_asig, { {} }, chan_trans, {}, {})"
```

```
47  fun tupac_step where
48    ‹tupac_step proc = (if proc = 0 then coordinator_step proc
      ↪  else participant_step)›
```

```
124 definition System  where
125 "System t P r = ((Coordinator 0 (t 0) (P ∪ {0}) r ‖ Channel)
    ↪  ‖ Participants P t)"
```

Wenn es einen Teilnehmer im Committed-Zustand gibt, so muss es eine Commit-Nachricht geben

```
8  definition inv1 where
9    ‹inv1 msgs states ⟷
10     ((∃proc a ack'. proc ≠ 0 ∧ states proc = Committed a
       ↪  ack') ⟶
11                  (∃sender rcpt. (sender, Send rcpt Commit) ∈
                    ↪  msgs))›
```

```
6  definition inv1 where
7    ‹inv1 s ⟷
8      ((∃msgs p. (snd s) p = PState PCommitted msgs) ⟶
9                  (∃sender rcpt. (sender, rcpt, Commit) ∈ snd
                   ↪  (fst s)))›
```

Wenn es eine Commit-Nachricht gibt oder der Koordinator im Committed-Zustand ist, so hat jeder Teilnehmer eine Yes-Nachricht gesendet

```
13  definition inv11 where
14    ‹inv11 msgs states procs ⟷
15      ((∃sender rcpt. (sender, Send rcpt Commit) ∈ msgs) ∨
        ↪  (∃all ack. states 0 = Committed all ack) ⟶
16                  (∀p ∈ procs. p ≠ 0 ⟶ (∃rcpt. (p, Send rcpt
                    ↪  Yes) ∈ msgs)))›
```

```
11  definition inv11 where
12    ‹inv11 procs s ⟷
13      ((∃sender rcpt. (sender, rcpt, Commit) ∈ snd (fst s)) ∨
        ↪  (∃all ack msgs. fst (fst s) = CState (CCommitted all
        ↪  ack) msgs) ⟶
14                  (∀p ∈ procs. p ≠ 0 ⟶ (∃rcpt. (p, rcpt,
                    ↪  Yes) ∈ snd (fst s))))›
```

Wenn der Koordinator im Collecting-Zustand ist, so stimmen hat er sich die korrekten Werte abgespeichert

```
22  definition inv13 where
23    ‹inv13 msgs states procs ⟷
24      (∀t all yes r. states 0 = Collecting t all yes r ⟶
25                  (procs = all ∧ (∀p ∈ yes. p ≠ 0 ⟶ (∃rcpt.
                    ↪  (p, Send rcpt Yes) ∈ msgs))))›
```

```
16  definition inv13 where
17    ‹inv13 procs s ⟷
18      (∀t all yes r msgs. fst (fst s) = CState (Collecting t
        ↪  all yes r) msgs ⟶
19                  (procs = all ∧ (∀p ∈ yes. p ≠ 0 ⟶ (∃rcpt.
                    ↪  (p, rcpt, Yes) ∈ snd (fst s)))))›
```

Wenn der Koordinator im Initial-Zustand ist, so stimmen hat er sich die korrekte Menge an Prozessen abgespeichert

```
27  definition inv14 where
28    ‹inv14 msgs states procs ⟷
29      (∀t all r. states 0 = Initial t all r ⟶ procs = all)›
```

```
21  definition inv14 where
22    ‹inv14 procs s ⟷
23      (∀t all r msgs. fst (fst s) = CState (CInitial t all r)
        ↪  msgs ⟶ procs = all)›
```

Wenn sich der Koordinator im Initial-, Collecting- oder Committed- Zustand befindet, so wurde keine Abort-Nachricht gesendet

```
31  definition inv15 where
32    ‹inv15 msgs states ⟷
33      ((∃t all yes r. states 0 = Initial t all r ∨ states 0 =
        ↪  Collecting t all yes r ∨ states 0 = Committed all
        ↪  yes) ⟶
34              (∀sender rcpt. (sender, Send rcpt Abort) ∉
                ↪  msgs))›
```

```
25  definition inv15 where
26    ‹inv15 s ⟷
27      ((∃t all yes r msgs. fst (fst s) = CState (CInitial t all
        ↪  r) msgs ∨ fst (fst s) = CState (Collecting t all yes
        ↪  r) msgs ∨ fst (fst s) = CState (CCommitted all yes)
        ↪  msgs) ⟶
28              (∀sender rcpt. (sender, rcpt, Abort) ∉ snd (fst
                ↪  s)))›
```

Wenn sich ein Teilnehmer im Aborted-Zustand befindet und keine Abort-Nachricht gesendet wurde, so hat der Teilnehmer auch keine Yes-Nachricht gesendet

```
40  definition inv17 where
41    ‹inv17 msgs states ⟷
42      (∀p. p ≠ 0 ∧ (∃all ack. states p = Aborted all ack) ∧
        ↪  (∀sender recpt. (sender, Send recpt Abort) ∉ msgs)
        ↪  ⟶ (∀rcpt. (p, Send rcpt Yes) ∉ msgs))›
```

```
30  definition inv17 where
31    ‹inv17 s ⟷
32      (∀p. p ≠ 0 ∧ (∃msgs. (snd s) p = PState PAborted msgs) ∧
        ↪  (∀sender recpt. (sender, recpt, Abort) ∉ snd (fst
        ↪  s)) ⟶ (∀rcpt. (p, rcpt, Yes) ∉ snd (fst s)))›
```

Wenn sich der Koordinator im Initial-, Collecting- oder Aborted- Zustand befindet, so wurde keine Commit-Nachricht gesendet

```
52  definition inv110 where
53    ‹inv110 msgs states ⟷
54      ((∃t all yes r. states 0 = Initial t all r ∨ states 0 =
        ↪  Collecting t all yes r ∨ states 0 = Aborted all yes)
        ↪  ⟶
55                (∀sender rcpt. (sender, Send rcpt Commit) ∉
                  ↪  msgs))›
```

```
34  definition inv110 where
35    ‹inv110 s ⟷
36      ((∃t all yes r msgs. fst (fst s) = CState (CInitial t all
        ↪  r) msgs ∨ fst (fst s) = CState (Collecting t all yes
        ↪  r) msgs ∨ fst (fst s) = CState (CAborted all yes)
        ↪  msgs) ⟶
37                (∀sender rcpt. (sender, rcpt, Commit) ∉ snd
                  ↪  (fst s)))›
```

Wenn eine Commit-Nachricht gesendet wurde, so befindet sich der Koordinator entweder im Committed- oder Forgotten-Zustand

```
57  definition inv111 where
58    ‹inv111 msgs states ⟷
59      ((∃sender rcpt. (sender, Send rcpt Commit) ∈ msgs) ⟶
        ↪  (∃all ack. states 0 = Committed all ack ∨ states 0 =
        ↪  Forgotten))›
```

```
39  definition inv111 where
40    ‹inv111 s ⟷
41      ((∃sender rcpt. (sender, rcpt, Commit) ∈ snd (fst s)) ⟶
42       (∃all ack cmsgs. fst (fst s) = CState (CCommitted all
         ↪  ack) cmsgs ∨ fst (fst s) = CState Forgotten cmsgs))›
```

Wenn eine Commit-Nachricht gesendet wurde, so wurde keine Abort-Nachricht gesendet

```
67  definition inv3 where
68    ‹inv3 msgs states ⟷
69      (∃sender recpt. (sender, Send recpt Commit) ∈ msgs) ⟶
        ↪  (∀sender recpt. (sender, Send recpt Abort) ∉ msgs)›
```

```
44  definition inv3 where
45    ‹inv3 s ⟷
46      (∃sender recpt. (sender, recpt, Commit) ∈ snd (fst s))
        ↪  ⟶ (∀sender recpt. (sender, recpt, Abort) ∉ snd (fst
        ↪  s))›
```

```
19  fun valid_event :: "('proc, 'msg) event ⇒ 'proc ⇒
20                      ('proc × ('proc, 'msg) send) set ⇒ bool"
                        ↪  where
21    "valid_event Start _ _ = True" |
22    "valid_event (Receive sender msg) proc msgs = ((sender,
      ↪  Send proc msg) ∈ msgs)" |
23    "valid_event _ _ _ = True"
24
25  inductive execute ::
26      "('proc, 'state, 'msg) step_func ⇒ ('proc ⇒ 'state) ⇒
       ↪  'proc set ⇒
27       ('proc × ('proc, 'msg) event) list ⇒
28       ('proc × ('proc, 'msg) send) set ⇒ ('proc ⇒ 'state) ⇒
         ↪  bool" where
29    "execute step init procs [] {} init" |
30    "⟦execute step init procs events msgs states;
31      proc ∈ procs;
32      valid_event event proc msgs;
33      step proc (states proc) event = (new_state, sent);
```

```
69  inductive reachable :: "('a, 's) ioa ⇒ 's ⇒ bool" for C ::
    ↪  "('a, 's) ioa"
70  where
71    reachable_0:  "s ∈ starts_of C ⟹ reachable C s"
72  | reachable_n:  "reachable C s ⟹ (s, a, t) ∈ trans_of C ⟹
    ↪  reachable C t"
73
74  definition invariant :: "[('a, 's) ioa, 's ⇒ bool] ⇒ bool"
75    where "invariant A P ⟷ (∀s. reachable A s ⟶ P s)"

380 lemma invariantI:
381   assumes "⋀s. s ∈ starts_of A ⟹ P s"
382     and "⋀s t a. reachable A s ⟹ P s ⟹ (s, a, t) ∈
        ↪  trans_of A ⟶ P t"
383   shows "invariant A P"
```

```
90  lemma invariant1:
91    assumes ‹execute tupac_step (λp. Initial (init_val p) UNIV
      ↪  r) UNIV events msgs' states'›
92    shows "inv1 msgs' states'"
```

```
369 lemma invariant1:
370   "invariant (System t (UNIV - {0}) r) inv1"
```

```
93    using assms proof(induction events arbitrary: msgs'
      ↪  states' r rule: List.rev_induct)
```

```
371  proof(induction rule: invariantI)
```

```
94      case Nil
95      then have statesInit:"∀p. states' p = Initial (init_val p)
        ↪  UNIV r"
96        using execute_init assms by fast
97      moreover have msgsEmpty:"msgs' = {}"
98        using execute_init Nil by fast
99      ultimately show ?case
100       using inv1_def by auto
```

```
372     case (1 s)
373     moreover have "starts_of (System t (UNIV - {0}) r) =
        ↪  {((CState (CInitial (t 0) UNIV r) {},{}),λp. PState
        ↪  (PInitial (t p)) {})}"
374       unfolding starts_of_def System_def par_def
          ↪  Coordinator_def Channel_def Participants_def by auto
375     ultimately have "s = ((CState (CInitial (t 0) UNIV r)
        ↪  {},{}),λp. PState (PInitial (t p)) {})"
376       by auto
377     then show ?case
378       unfolding inv1_def by simp
```

```
102     case (snoc x events)
103     obtain proc event where "x = (proc, event)"
104       by fastforce
105     hence exec: "execute tupac_step (λp. Initial (init_val p)
        ↪  UNIV r) UNIV
106                   (events @ [(proc, event)]) msgs' states'"
107       using snoc.prems assms by fast
108     from this obtain msgs states sent new_state
109       where step_rel1: "execute tupac_step (λp. Initial
          ↪  (init_val p) UNIV r) UNIV events msgs states"
110         and step_rel2: "tupac_step proc (states proc) event =
            ↪  (new_state, sent)"
111         and step_rel3: "msgs' = msgs ∪ ((λmsg. (proc, msg)) `
            ↪  sent)"
112         and step_rel4: "states' = states (proc := new_state)"
113       by auto
```

```
380     case (2 s t a)
381     then show ?case
382       using invariant1_trans by metis
```

```
114     show ?case
115     proof (cases "proc = 0")
116       case True
117       then have "coordinator_step proc (states 0) event =
          ↪  (new_state, sent)"
118         using step_rel2 by simp
119       moreover have"inv1 msgs states"
120         using snoc.IH step_rel1 by fast
121       ultimately show ?thesis
122         using invariant1_coordinator True step_rel3 step_rel4
            ↪  exec by metis
```

```
380     case (2 s t a)
381     then show ?case
382       using invariant1_trans by metis
```

```
124       case False
125       then have "participant_step (states proc) event =
          ↪  (new_state, sent)"
126         using step_rel2 by simp
127       moreover have"inv1 msgs states"
128         using snoc.IH step_rel1 by fast
129       ultimately show ?thesis
130         using invariant1_participant False step_rel3 step_rel4
            ↪  exec by metis
131     qed
132   qed
```

```
380     case (2 s t a)
381     then show ?case
382       using invariant1_trans by metis
```

```
5  lemma invariant1_coordinator:
6    assumes "coordinator_step 0 (states 0) event = (new_state,
       ↪  sent)"
7      and "msgs' = msgs ∪ ((λmsg. (0, msg)) ` sent)"
8      and "states' = states (0 := new_state)"
9      and "execute tupac_step (λp. Initial (init_val p) UNIV r)
         ↪  UNIV (events @ [(0, event)]) msgs' states'"
10     and "inv1 msgs states"
11   shows "inv1 msgs' states'"
```

```
63 lemma invariant1_trans:
64   assumes "reachable (System t (UNIV - {0}) r) s"
65     and "inv1 s"
66     and "(s, a, s') ∈ trans_of (System t (UNIV - {0}) r)"
67   shows "inv1 s'"
```

```
12    using assms apply (induction rule: coordinator_induct)
13    using inv1_def UnCI assms(2,3,5) apply (metis (mono_tags,
       ↪  lifting) fun_upd_apply)+
14    done
```

```
68  proof -
```

```
16  lemma invariant1_participant:
17    assumes "participant_step (states proc) event =
      ↪  (new_state, sent)"
18      and "proc ≠ 0"
19      and "msgs' = msgs ∪ ((λmsg. (proc, msg)) ` sent)"
20      and "states' = states (proc := new_state)"
21      and "execute tupac_step (λp. Initial (init_val p) UNIV r)
        ↪  UNIV (events @ [(proc, event)]) msgs' states'"
22      and "inv1 msgs states"
23    shows "inv1 msgs' states'"
24    using assms
25  proof (induction rule: participant_induct)
```

```
63  lemma invariant1_trans:
64    assumes "reachable (System t (UNIV - {0}) r) s"
65      and "inv1 s"
66      and "(s, a, s') ∈ trans_of (System t (UNIV - {0}) r)"
67    shows "inv1 s'"
68  proof -
```

```
25  proof (induction rule: participant_induct)
```

```
69    obtain cstate cmsgs chmsgs pstates where decomp_s: "s =
      ↪  ((CState cstate cmsgs, chmsgs), pstates)"
70      by (metis coord_state.exhaust surj_pair)
71    obtain cstate' cmsgs' chmsgs' pstates' where decomp_s': "s'
      ↪  = ((CState cstate' cmsgs', chmsgs'), pstates')"
72      by (metis coord_state.exhaust surj_pair)
```

```
25  proof (induction rule: participant_induct)
```

```
73    show ?thesis
74      using assms(3) unfolding decomp_s decomp_s'
75    proof (induction rule: System_trans_inv)
76      case System_Step
77      then show ?case
78      proof(cases a)
```

```
25  proof (induction rule: participant_induct)
```

```
79          case (Start rcpt)
80          then have "pstates = pstates'"
81           using System_Step(3) unfolding parts_trans_def by auto
82          moreover have "chmsgs = chmsgs'"
83            using Start System_Step(2) by auto
```

```
25  proof (induction rule: participant_induct)
```

```
84          ultimately have "inv1 s ⟷ inv1 s'"
85            using inv1_def by (metis assms(2) decomp_s decomp_s'
                ↪  fst_conv snd_conv)
86          then show ?thesis
87            using assms(2) decomp_s decomp_s' by simp
```

```
25  proof (induction rule: participant_induct)
```

```
89          case (Send sender rcpt _)
90        then have "(∃msgs p. pstates p = PState PCommitted msgs)
           ↪  ⟷ (∃msgs p. pstates' p = PState PCommitted msgs)"
91          by (metis System_Step.hyps(3) part_state.exhaust
             ↪  parts_trans_Send)
```

```
25  proof (induction rule: participant_induct)
```

```
92          moreover have "(∃sender rcpt. (sender, rcpt, Commit) ∈
            ↪  chmsgs) ⟶ (∃sender rcpt. (sender, rcpt, Commit) ∈
            ↪  chmsgs')"
93            using Send System_Step(2) by auto
94          ultimately have "inv1 s ⟷ inv1 s'"
95            using inv1_def by (metis assms(2) decomp_s decomp_s'
               ↪  fst_conv snd_conv)
96          then show ?thesis
97            using assms(2) decomp_s decomp_s' by simp
```

```
25  proof (induction rule: participant_induct)
```

```
99          case (Receive sender rcpt msg)
100         have chEquiv:"chmsgs = chmsgs'"
101           using Receive System_Step(2) by blast
102         moreover have receive:"(sender, rcpt, msg) ∈ chmsgs'"
103           by (metis Receive decomp_s decomp_s'
              ↪  System_Receive_chan_step assms(3))
104         then show ?thesis
105         proof(cases "rcpt = 0")
```

```
25  proof (induction rule: participant_induct)
```

```
106         case True
107         then have "pstates = pstates'"
108           using True System_Step(3) Receive unfolding
              ↪  parts_trans_def by auto
109         then have "inv1 s ⟷ inv1 s'"
110           using inv1_def chEquiv by (metis decomp_s decomp_s'
              ↪  fst_conv snd_conv)
111         then show ?thesis
112           using assms(2) decomp_s decomp_s' by simp
```

```
25  proof (induction rule: participant_induct)
```

```
114         case False
115         then have otherPartsEquiv:"∀p ≠ rcpt. pstates p =
            ↪  pstates' p"
116           using System_Step(3) Receive unfolding
              ↪  parts_trans_def by auto
```

```
25  proof (induction rule: participant_induct)
```

```
118           proof(cases "pstates rcpt = pstates' rcpt")
119             case True
120             then have "pstates = pstates'"
121               using otherPartsEquiv by auto
122             then show ?thesis
123               using chEquiv assms(2) inv1_def decomp_s
                  ↪  decomp_s' by (metis fst_conv snd_conv)
```

```
25  proof (induction rule: participant_induct)
```

```
125             case False
126             then obtain pstate pmsgs where stateDecomp:"pstates
                ↪  rcpt = PState pstate pmsgs"
127               using part_state.exhaust by blast
128             then obtain new_state sent where
                ↪  step:"participant_step rcpt pstate a =
                ↪  (new_state, sent)"
129               using System_Step(3) False by fastforce
130             have update_eq: "pstates' rcpt = PState new_state
                ↪  (pmsgs ∪ ((λmsg. (rcpt, msg)) ` sent))"
131               by (smt (verit, ccfv_SIG) step parts_trans_proj
                   ↪  stateDecomp parts_trans_step_result
                   ↪  System_Step(3) False Receive)
```

"By itself, the I/O automaton model has very little structure, which allows it to be used for modelling many different types of distributed systems. Additional structure must be added to the basic model to enable it to describe particular types of asynchronous systems." Lynch buch s.199

blblblb